

KICD-CDS-서술형-25문제

문항 1:

Azure Policy와 Azure RBAC(역할 기반 액세스 제어)의 주요 차이점은 무엇이며, 언제 각각을 사용해야 하나요?

Azure Policy와 Azure RBAC(역할 기반 액세스 제어)는 Azure 거버넌스의 핵심 요소이지만, 서로 다른 목적을 수행하며 상호 보완적으로 사용됩니다. 이 둘의 근본적인 차이는 '무엇을' 제어하는가와 '누가' 제어하는가에 있습니다.

- **Azure RBAC (Role-Based Access Control):** RBAC는 **사용자 작업(Actions)**에 중점을 둡니다. 즉, 특정 사용자, 그룹 또는 서비스 주체가 Azure 리소스에 대해 수행할 수 있는 작업(예: 생성, 읽기, 업데이트, 삭제)을 정의하고 관리합니다. RBAC는 '누가 이 가상 머신을 시작/중지할 수 있는가?' 또는 '어떤 팀이 이 리소스 그룹에 리소스를 배포할 수 있는가?'와 같은 권한 부여 질문에 답합니다.1 본질적으로 RBAC는 Azure의 보안 주체(Principal)에게 역할을 할당하여 특정 범위(Scope) 내에서 권한을 부여하는 인증 및 권한 부여 시스템입니다.
- **Azure Policy:** 반면, Azure Policy는 **리소스 속성(Properties)**과 **규정 준수(Compliance)**에 중점을 둡니다. 이는 리소스가 배포되거나 업데이트될 때 조직의 표준, 보안 요구 사항 및 규정을 준수하도록 보장하는 규칙 엔진입니다. Policy는 '모든 스토리지 계정은 암호화가 활성화되어야 한다' 또는 '리소스는 승인된 Azure 지역에만 배포될 수 있다'와 같은 규칙을 강제합니다.2 Policy는 리소스의 구성 상태를 평가하고, 비준수 리소스를 보고하거나, 심지어 비준수 리소스의 생성을 차단할 수도 있습니다.

이 둘의 관계를 명확히 이해하는 것은 효과적인 클라우드 거버넌스 전략의 핵심입니다. 예를 들어, RBAC는 특정 개발자 그룹에게 스토리지 계정을 생성할 수 있는 권한을 부여할 수 있습니다. 그러나 Azure Policy는 해당 그룹이 스토리지 계정을 생성할 때 반드시 '저장 데이터 암호화' 옵션을 활성화하도록 강제할 수 있습니다. 만약 개발자가 이 정책을 위반하는 스토리지 계정을 생성하려고 시도하면, Azure Policy는 **Deny** 효과를 통해 해당 작업을 차단할 수 있습니다.

이러한 작동 방식은 Azure Policy가 단순한 사후 감사 도구가 아니라, **사전 예방적 거버넌스 메커니즘**을 보여줍니다. Azure Resource Manager(ARM)를 통해 리소스 생성 또는 수정 요청이 제출되면, Azure Policy 엔진이 이를 가로채어 할당된 정책에 대해 평가합니다. **Deny** 정책을 위반하는 요청은 배포 프로세스가 시작되기도 전에 거부됩니다.2 이는 인프라 배포 파이프라인의 초기 단계에서부터 규정 준수를 강제하는 'Shift-Left' 거버넌스 접근 방식을 가능하게 하여, 잘못된 구성으로 인한 문제를 원천적으로 방지합니다.

사용 시나리오:

- **Azure RBAC 사용 시기:**
 - 사용자 또는 팀의 책임에 따라 Azure 리소스에 대한 액세스 권한을 분리해야 할 때 (예: 네트워크 팀은 가상 네트워크만 관리하고, 데이터베이스 팀은 SQL 데이터베이스만 관리).
 - 최소 권한 원칙(Principle of Least Privilege)을 구현하여 사용자에게 업무 수행에 필요한 최소한의 권한만 부여하고자 할 때.
 - 특정 관리 작업을 특정 관리자 그룹에게만 위임해야 할 때.
- **Azure Policy 사용 시기:**
 - 모든 Azure 리소스에 대해 일관된 태그(예: CostCenter, Environment)를 강제하여 비용 추적 및 관리를 용이하게 하고자 할 때.

- HIPAA, PCI-DSS 등과 같은 산업 규정 준수 요구 사항을 충족하기 위해 특정 보안 구성(예: 모든 VM 디스크 암호화)을 강제해야 할 때.
- 비용 통제를 위해 고가의 VM SKU나 특정 서비스의 배포를 제한하고자 할 때.
- Azure Arc를 통해 온프레미스 및 멀티클라우드 환경까지 거버넌스 규칙을 일관되게 확장하고자 할 때.3

결론적으로, RBAC는 **행위자 중심(Actor-focused)**의 권한 관리 도구이며, Azure Policy는 **리소스 중심(Resource-focused)**의 규정 준수 및 거버넌스 도구입니다. 이 두 가지를 함께 사용하면 Azure 환경에 대한 포괄적이고 심층적인 제어 체계를 구축할 수 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/governance/policy/overview>

문항 2:

Azure Policy 정의(Definition)의 기본 구조와 각 구성 요소(mode, parameters, policyRule)의 역할은 무엇입니까?

Azure Policy 정의는 JSON 형식을 사용하여 리소스가 준수해야 할 조건과 해당 조건이 충족될 때 수행할 효과(Effort)를 기술합니다. 이 정의는 Azure 환경 전체에 일관된 규칙을 적용하는 기초가 됩니다. 정책 정의의 핵심 구조는 `properties` 객체 내에 있으며, 주요 구성 요소는 `displayName`, `description`, `mode`, `metadata`, `parameters`, `policyRule` 입니다.4

1. `displayName` 및 `description`:

- `displayName` (최대 128자) 및 `description` (최대 512자)은 정책 정의를 식별하고 해당 정책이 사용되는 컨텍스트를 제공하는 데 사용됩니다. 사람이 읽을 수 있는 이름과 설명을 통해 정책의 목적을 명확히 전달합니다.4

2. `mode`:

- `mode`는 정책이 어떤 종류의 Azure 리소스를 평가할지 결정합니다. 이는 정책의 적용 범위를 최적화하고 관련 없는 리소스가 비준수로 표시되는 것을 방지하는 데 중요합니다.
- `all`: 리소스 그룹, 구독 및 모든 리소스 유형을 평가합니다. 가장 포괄적인 모드입니다.
- `indexed`: 태그(tags)와 위치(location)를 지원하는 리소스 유형만 평가합니다. 태그나 위치 관련 정책을 만들 때 이 모드를 사용하는 것이 좋습니다. 이렇게 하면 태그나 위치 속성이 없는 리소스(예: 일부 네트워크 구성 요소)가 불필요하게 비준수로 보고되는 것을 방지할 수 있습니다.4
- **기타 리소스 공급자 모드**: `Microsoft.Kubernetes.Data`, `Microsoft.KeyVault.Data`, `Microsoft.Network.Data`와 같이 특정 서비스에 대한 심층적인 거버넌스를 제공하는 특수 모드도 있습니다. 예를 들어, `Microsoft.Kubernetes.Data` 모드는 Azure Kubernetes Service(AKS) 클러스터 내부의 Pod, Ingress와 같은 구성 요소를 관리하는 데 사용됩니다.

3. `parameters`:

- `parameters`는 정책 정의를 더욱 유연하고 재사용 가능하게 만듭니다. 매개변수를 사용하면 정책 규칙 내에서 하드코딩된 값을 사용하는 대신, 정책을 할당할 때 동적으로 값을 제공할 수 있습니다.1
- 예를 들어, '허용된 위치' 정책을 정의할 때 특정 위치 목록을 하드코딩하는 대신, `allowedLocations`라는 이름의 `array` 타입 매개변수를 만들 수 있습니다. 이렇게 하면 동일한 정책 정의를 사용하여 한 구독에는 `[ "koreacentral", "koreasouth" ]`를, 다른 구독에는 `[ "japaneast", "japanwest" ]`를 할당할 수 있습니다.
- 각 매개변수는 `type` (예: `string`, `array`, `integer`)과 선택적으로 `metadata` (설명, 표시 이름 등) 및 `defaultValue`를 가집니다.4

4. `policyRule`:

- `policyRule` 은 정책의 핵심 로직을 포함하며, `if` 블록과 `then` 블록으로 구성됩니다.
- **if 블록**: 평가할 조건을 정의합니다. 하나 이상의 논리적 연산자(`allOf`, `anyOf`, `not`)를 사용하여 복잡한 조건을 구성할 수 있습니다. 조건은 리소스 속성 필드(별칭(alias)을 통해 접근)를 특정 값과 비교합니다. 예를 들어, `"field": "location", "notIn": "[parameters('allowedLocations')]"` 와 같은 조건은 리소스의 위치가 매개변수로 전달된 허용된 위치 목록에 없는 경우 `true` 로 평가됩니다.⁴
- **then 블록**: `if` 블록의 조건이 `true` 로 평가될 때 수행할 `effect` (효과)를 정의합니다. 주요 효과는 다음과 같습니다:
 - **Deny**: 리소스 생성 또는 업데이트를 차단합니다.
 - **Audit**: 리소스 작업을 허용하지만, 규정 준수 대시보드에 비준수로 기록합니다.
 - **Append**: 리소스 생성/업데이트 시 정의된 필드와 값을 추가합니다 (예: 누락된 태그 추가).
 - **Modify**: 리소스 생성/업데이트 시 태그나 속성을 추가하거나 업데이트합니다.
 - **DeployIfNotExists**: 조건에 맞는 리소스가 존재하지 않을 경우, 관련 리소스를 배포합니다 (예: VM에 대한 진단 설정이 없으면 배포).
 - **AuditIfNotExists**: 조건에 맞는 리소스가 존재하지 않을 경우, 비준수로 보고합니다.

이러한 구성 요소들을 조합하여 조직의 특정 거버넌스 요구사항에 맞는 강력하고 유연한 정책을 만들 수 있습니다. 예를 들어, `indexed` 모드를 사용하고 위치를 확인하는 `policyRule` 을 가진 정책은 태그와 위치를 지원하는 리소스에 대해서만 효율적으로 규정 준수를 강제할 수 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/governance/policy/concepts/definition-structure>

문항 3:

Azure Policy의 이니셔티브(Initiative)란 무엇이며, 이를 사용하면 어떤 이점이 있습니까?

Azure Policy의 이니셔티브(Initiative), 또는 정책 세트(Policy Set)는 단일 목표를 달성하기 위해 함께 그룹화된 **정책 정의(Policy Definitions)의 모음**입니다. 개별 정책이 특정 규칙(예: '특정 VM SKU만 허용')을 강제하는 반면, 이니셔티브는 여러 관련 정책을 하나의 단위로 묶어 더 광범위한 거버넌스 목표(예: 'PCI-DSS 규정 준수')를 달성하도록 돕습니다.¹

예를 들어, 조직의 '기본 보안 태세 강화'라는 이니셔티브는 다음과 같은 여러 개별 정책 정의를 포함할 수 있습니다:

- 모든 스토리지 계정에 대해 보안 전송(HTTPS) 요구
- 가상 머신에 대한 네트워크 보안 그룹(NSG) 규칙 감사
- SQL 데이터베이스에 대한 투명한 데이터 암호화(TDE) 활성화
- 모든 리소스에 'Owner' 태그 요구

이러한 정책들을 개별적으로 할당하고 관리하는 대신, '기본 보안 태세 강화' 이니셔티브 하나를 구독이나 관리 그룹에 할당할 수 있습니다.

이니셔티브 사용의 주요 이점:

1. **관리 간소화 및 일관성:** 여러 정책을 개별적으로 추적하고 할당하는 것은 복잡하고 오류가 발생하기 쉽습니다. 이니셔티브는 관련 정책들을 하나의 논리적 단위로 묶어주므로, 할당, 모니터링, 업데이트가 훨씬 간편해집니다. 이니셔티브 하나만 할당하면 그 안에 포함된 모든 정책이 대상 범위에 일관되게 적용됩니다.³
2. **규정 준수 목표 달성 용이:** Azure는 PCI-DSS, ISO 27001, NIST SP 800-53 등과 같은 일반적인 규정 준수 표준에 맞춰 미리 정의된 **기본 제공(Built-in) 이니셔티브**를 제공합니다. 이를 통해 조직은 복잡한 규제 요구사항을 충족하기 위해 어떤 정책이 필요한지 직접 파악할 필요 없이, 해당 이니셔티브를 할당하기만 하면 됩니다. 이는 규정 준수 감사를 준비하는 데 드는 시간과 노력을 크게 절감시켜 줍니다.
3. **계층적 구조 및 매개변수 관리:** 이니셔티브는 자체적으로 매개변수를 가질 수 있습니다. 이니셔티브 수준에서 정의된 매개변수는 해당 이니셔티브 내의 여러 정책 정의에 값을 전달할 수 있습니다. 예를 들어, 이니셔티브에 `allowedLocations` 라는 매개변수를 만들고, 이 값을 이니셔티브 내의 'VM 위치 제한' 정책과 '스토리지 위치 제한' 정책 모두에 전달할 수 있습니다. 이렇게 하면 할당 시 단일 매개변수 값만 설정하여 여러 정책을 일관되게 구성할 수 있습니다.
4. **포괄적인 규정 준수 평가:** 이니셔티브를 할당하면 Azure Policy의 규정 준수 대시보드에서 전체 이니셔티브에 대한 집계된 준수 상태를 볼 수 있습니다. 이를 통해 'PCI-DSS 이니셔티브의 준수율은 85%'와 같이 상위 수준의 거버넌스 목표 달성 현황을 한눈에 파악할 수 있으며, 비준수 항목을 드릴다운하여 개별 정책 및 리소스 수준에서 문제를 해결할 수 있습니다.³

결론적으로, 이니셔티브는 정책 관리를 추상화하고 단순화하는 강력한 도구입니다. 단일 정책으로 미세한 제어를 수행하고, 이니셔티브를 사용하여 이러한 제어들을 그룹화함으로써 조직은 확장 가능하고 관리하기 쉬운 방식으로 Azure 환경 전반에 걸쳐 포괄적인 거버넌스 전략을 구현할 수 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/governance/policy/concepts/initiative-definition-structure>

문항 4:

Azure Blueprints란 무엇이며, ARM 템플릿과 어떻게 다릅니까?

Azure Blueprints는 조직의 표준, 패턴 및 요구 사항을 준수하는 클라우드 환경의 반복 가능한 배포를 가능하게 하는 **선언적 패키징 서비스**입니다. Blueprint는 단순히 리소스를 배포하는 것을 넘어, 규정 준수 및 거버넌스가 내장된 완전한 환경을 정의하고 프로비저닝하는 데 중점을 둡니다.⁵

Azure Blueprint는 다음과 같은 다양한 **아티팩트(Artifacts)**를 하나의 패키지로 묶습니다 ⁶:

- **리소스 그룹(Resource Groups):** 환경의 논리적 구조를 정의합니다.
- **ARM 템플릿(Azure Resource Manager templates):** 가상 머신, 네트워크, 데이터베이스 등 배포할 Azure 리소스를 정의합니다.
- **정책 할당(Policy Assignments):** 배포된 환경에 적용될 거버넌스 규칙(예: 특정 지역에만 배포 허용)을 정의합니다.
- **역할 할당(Role Assignments):** 배포된 리소스에 대한 액세스 제어(RBAC)를 정의합니다 (예: 특정 그룹에 '기여자' 역할 부여).

ARM 템플릿과의 주요 차이점:

ARM 템플릿과 Azure Blueprints는 모두 코드형 인프라(Infrastructure as Code)를 구현하는 도구이지만, 그 범위와 목적에서 근본적인 차이가 있습니다. 이 둘의 관계는 "집 설계도"와 "주택 단지 종합 계획"에 비유할 수 있습니다.⁶

특징	ARM 템플릿	Azure Blueprints
목적	특정 Azure 리소스 집합의 배포	거버넌스가 포함된 완전한 환경의 조성 및 통제
비유	개별 주택의 설계도 (The house plans)	도로, 공원, 건축 규제를 포함한 주택 단지 종합 계획 (The master plan for the housing complex)
구성요소	리소스 정의, 매개변수, 변수, 출력	ARM 템플릿, 정책 할당, 역할 할당, 리소스 그룹 등 다양한 아티팩트의 집합
상태관리	배포 후 ARM 템플릿과 배포된 리소스 간의 활성 관계가 없음 . 템플릿을 다시 실행하면 리소스가 업데이트될 뿐, 지속적인 연결은 유지되지 않음.	Blueprint 할당과 배포된 리소스 간에 지속적인 관계가 유지됨 . 이를 통해 배포된 환경의 준수 상태를 추적하고 감사할 수 있음.
리소스 잠금	기능 없음	리소스 잠금(Resource Locking) 기능을 제공하여 Blueprint에 의해 배포된 리소스를 의도하지 않은 변경이나 삭제로부터 보호할 수 있음. ⁷
범위	리소스 그룹 수준에서 배포됨.	구독 또는 관리 그룹 수준에서 정의 되고, 구독 수준에서 할당 됨.
사용사례	단일 애플리케이션 스택 또는 특정 워크로드 배포.	VDI 환경, 보안 및 규정 준수가 중요한 워크로드(예: HIPAA, PCI), 개발/테스트 환경 등 표준화된 환경을 반복적으로 배포.

가장 중요한 차이점 중 하나는 **지속적인 관계**와 **리소스 잠금**입니다. ARM 템플릿으로 리소스를 배포하면, 그 이후 템플릿과 리소스는 독립적입니다. 누군가 포털에서 리소스를 수정해도 템플릿에는 반영되지 않습니다. 하지만 Blueprint를 사용하여 환경을 배포하면, Azure는 Blueprint 할당과 배포된 리소스 간의 연결을 유지합니다. 이를 통해 Blueprint에서 정의한 거버넌스(정책, RBAC)가 계속해서 적용되고 있는지 추적할 수 있습니다.

또한, Blueprint의 **리소스 잠금** 기능은 거버넌스를 한 단계 더 강화합니다. '읽기 전용' 또는 '삭제 금지' 잠금을 설정하여 Blueprint의 일부로 배포된 핵심 인프라가 실수로 또는 악의적으로 변경되는 것을 방지할 수 있습니다. 구독 소유자조차도 이 잠금을 해제하려면 먼저 Blueprint 할당에서 잠금을 제거해야 합니다.⁵

결론적으로, ARM 템플릿은 '무엇을' 배포할지에 대한 강력한 도구인 반면, Azure Blueprints는 '어떻게' 배포하고 '어떻게' 관리할지에 대한 거버넌스 프레임워크를 함께 제공하는 포괄적인 환경 조성 도구입니다. 조직이 표준화되고 규정을 준수하는 환경을 대규모로 신속하게 프로비저닝해야 할 때 Azure Blueprints는 필수적인 서비스입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/governance/blueprints/overview>

문항 5:

Azure Arc의 핵심 기능은 무엇이며, 하이브리드 및 멀티클라우드 환경에서 어떤 가치를 제공합니까?

Azure Arc는 Azure의 관리 및 거버넌스 기능을 Azure 데이터센터 외부로 확장하여, 온프레미스, 엣지, 그리고 다른 클라우드(예: AWS, GCP) 환경에 있는 리소스까지 일관되게 관리할 수 있도록 하는 **단일 제어 평면(Single Control Plane)**을 제공하는 서비스입니다.⁸ 본질적으로 Azure Arc는 기존의 비-Azure 리소스를 Azure Resource Manager(ARM)에 '투영(project)'하여 마치 네이티브 Azure 리소스처럼 관리할 수 있게 만드는 다리 역할을 합니다.

Azure Arc의 핵심 기능:

Azure Arc는 다양한 유형의 리소스를 지원하며, 각 유형에 맞는 관리 기능을 제공합니다.

1. Azure Arc-enabled Servers:

- 온프레미스(Windows, Linux) 또는 다른 클라우드에 있는 물리적 서버와 가상 머신을 Azure에 연결합니다.
- 연결된 서버에 Azure Policy를 적용하여 거버넌스를 강제하고, Azure Monitor를 사용하여 모니터링하며, Microsoft Defender for Cloud로 보안을 강화할 수 있습니다.
- Azure VM 확장(Extensions)을 사용하여 Azure Automation, Log Analytics 에이전트 등과 같은 Azure 관리 서비스를 배포하고 구성할 수 있습니다.⁸

2. Azure Arc-enabled Kubernetes:

- 어디서든 실행되는 모든 CNCF(Cloud Native Computing Foundation) 인증 Kubernetes 클러스터를 Azure에 연결하고 구성할 수 있습니다.
- GitOps를 사용하여 Git 리포지토리에서 클러스터 구성을 대규모로 배포하고 관리할 수 있습니다. 이는 모든 연결된 클러스터가 코드에 정의된 원하는 상태를 유지하도록 보장하는 강력한 기능입니다.¹⁰
- Azure Policy for Kubernetes를 사용하여 클러스터 전반에 걸쳐 보안 및 규정 준수 정책(예: 특정 레지스트리에서만 이미지 풀 허용)을 적용할 수 있습니다.

3. Azure Arc-enabled Data Services:

- 온프레미스, 엣지, 멀티클라우드 환경의 Kubernetes 클러스터 위에서 Azure SQL Managed Instance나 PostgreSQL과 같은 Azure PaaS 데이터 서비스를 실행할 수 있습니다.
- 이를 통해 데이터 주권(Data Sovereignty)이나 규제 요구사항, 또는 낮은 지연 시간(low-latency) 요구사항으로 인해 데이터를 특정 위치에 두어야 하는 경우에도 Azure의 관리형 데이터 서비스를 활용할 수 있습니다.⁸

하이브리드 및 멀티클라우드 환경에서의 가치:

Azure Arc는 기존의 복잡한 하이브리드 및 멀티클라우드 관리를 단순화하고 통합함으로써 상당한 가치를 제공합니다.

- 통합된 거버넌스 및 규정 준수:** 조직은 Azure Policy를 사용하여 Azure, 온프레미스, 멀티클라우드 환경에 걸쳐 일관된 보안 및 구성 정책을 적용할 수 있습니다. 이는 분산된 환경에서 일관된 거버넌스를 유지하는 데 따르는 어려움을 해결해 줍니다.³
- 단일화된 관리 및 가시성:** Azure Portal, Azure CLI, Azure PowerShell 등 익숙한 Azure 도구를 사용하여 모든 리소스를 한 곳에서 보고 관리할 수 있습니다. Azure Resource Graph를 통해 모든 Arc-enabled 리소스를 포함한 전체 환경에 대한 인벤토리 쿼리를 실행할 수 있어 가시성이 크게 향상됩니다.⁸
- DevOps 프랙티스 확장:** GitOps와 같은 최신 DevOps 프랙티스를 온프레미스 Kubernetes 클러스터까지 확장하여 애플리케이션 및 구성 배포를 자동화하고 일관성을 유지할 수 있습니다.¹¹

- **Azure 서비스의 유연한 소비:** Azure Arc는 기술적인 도구를 넘어 Microsoft의 전략적인 상업적 통로이기도 합니다. 기업들은 이미 보유한 인프라 위에서 Azure의 관리 및 데이터 서비스를 소비하고 Azure 구독을 통해 비용을 지불할 수 있습니다. 예를 들어, 온프레미스 서버에 대해 Microsoft Defender for Cloud를 활성화하거나, 기존의 Windows Server/SQL Server 라이선스를 Azure Arc를 통해 유연한 종량제(Pay-as-you-go) 모델로 전환할 수 있습니다.¹¹ 이는 클라우드로의 전면적인 마이그레이션 없이도 Azure 서비스의 이점을 누릴 수 있게 하여, Azure 생태계로의 진입 장벽을 낮추고 Microsoft의 수익 모델을 자체 데이터센터 너머로 확장하는 효과를 가져옵니다.

결론적으로, Azure Arc는 분산된 IT 환경을 가진 조직이 중앙 집중적이고 일관된 방식으로 전체 자산을 관리, 통제 및 보호할 수 있도록 지원하는 핵심적인 하이브리드 및 멀티클라우드 솔루션입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-arc/overview>

문항 6:

Azure Arc-enabled Kubernetes에서 GitOps는 어떻게 작동하며, 이를 통해 얻을 수 있는 이점은 무엇입니까?

Azure Arc-enabled Kubernetes에서 GitOps는 클러스터 구성 및 애플리케이션 배포를 자동화하고 관리하기 위한 핵심적인 운영 모델입니다. GitOps는 **Git 리포지토리를 단일 진실 공급원(Single Source of Truth)**으로 사용하여 Kubernetes 클러스터의 원하는 상태(Desired State)를 선언적으로 정의합니다. Azure Arc는 클러스터에 배포된 에이전트(Flux 기반)를 통해 이 모델을 구현합니다.¹⁰

GitOps 작동 방식:

1. **상태 정의:** 개발자나 운영자는 Kubernetes 매니페스트(YAML 파일), Helm 차트, Kustomize 파일 등을 사용하여 애플리케이션과 클러스터 구성의 원하는 상태를 정의하고 이를 Git 리포지토리에 커밋합니다. 이 리포지토리는 모든 환경 구성의 '진실'이 됩니다.
2. **구성 리소스 생성:** Azure Portal, CLI 등을 사용하여 Azure Arc로 연결된 Kubernetes 클러스터에 `Microsoft.KubernetesConfiguration/fluxConfigurations` 타입의 구성 리소스를 생성합니다. 이 리소스는 추후 적절한 Git 리포지토리의 URL, 브랜치, 경로 및 동기화 간격과 같은 정보를 담고 있습니다.¹⁰
3. **에이전트 배포 및 동기화:** 구성 리소스가 생성되면, Azure Arc는 대상 Kubernetes 클러스터의 `cluster-config` 네임스페이스에 Flux 컨트롤러와 같은 GitOps 에이전트를 배포합니다.
4. **지속적인 조정(Reconciliation):** 클러스터 내에서 실행되는 이 에이전트는 주기적으로 Git 리포지토리를 폴링하여 리포지토리에 정의된 상태와 클러스터의 실제 상태(Actual State)를 비교합니다.
5. **자동 배포 및 드리프트 수정:** 만약 두 상태 간에 차이(Drift)가 발견되면, 에이전트는 Git 리포지토리의 최신 커밋을 클러스터에 적용(pull)하여 실제 상태를 원하는 상태와 일치시킵니다. 예를 들어, 개발자가 Git 리포지토리에서 컨테이너 이미지 버전을 업데이트하면, 에이전트가 이를 감지하고 클러스터의 배포를 자동으로 업데이트합니다. 마찬가지로, 누군가 `kubectl`을 사용하여 수동으로 클러스터의 상태를 변경하면, 에이전트는 다음 동기화 주기 때 이를 원래의 선언된 상태로 되돌려 놓습니다.

GitOps의 이점:

- **일관성 및 표준화:** 모든 구성이 Git에 중앙 집중화되어 있으므로, 수십 또는 수백 개의 Kubernetes 클러스터에 걸쳐 일관된 구성을 쉽게 적용하고 유지할 수 있습니다. 이는 '구성 드리프트(Configuration Drift)' 문제를 해결하는 데 매우 효과적입니다.⁸

- **향상된 보안:** 개발자는 Kubernetes API 서버에 직접 접근할 필요가 없습니다. 대신, Git 리포지토리에 변경 사항을 커밋하고 Pull Request(PR) 프로세스를 통해 동료의 검토와 승인을 받습니다. 에이전트가 클러스터 내에서 Git 리포지토리로부터 변경 사항을 가져오는 '풀(pull)' 모델은, 외부 시스템이 클러스터에 직접 명령을 내리는 '푸시(push)' 모델보다 안전합니다.
- **감사 용이성 및 버전 관리:** Git의 커밋 기록은 클러스터에 대한 모든 변경 사항의 완전한 감사 추적(Audit Trail)을 제공합니다. 누가, 언제, 무엇을 변경했는지 명확하게 알 수 있습니다. 또한, 문제가 발생했을 때 이전의 안정적인 상태로 쉽게 되돌릴 수 있는 롤백 기능을 제공합니다.
- **개발자 생산성 향상:** 개발팀은 이미 익숙한 Git 워크플로우와 도구(예: PR, CI/CD 파이프라인)를 사용하여 애플리케이션을 프로덕션에 배포하고 업데이트할 수 있습니다. 인프라팀과 별도의 복잡한 배포 도구나 절차가 필요 없어 개발 속도가 빨라집니다.¹¹
- **대규모 자동화:** Azure Policy와 결합하여, 새로운 Azure Arc-enabled Kubernetes 클러스터가 추가될 때마다 자동으로 특정 GitOps 구성을 적용하도록 정책을 설정할 수 있습니다. 이는 제로 터치(Zero-touch) 프로비저닝 및 구성을 가능하게 하여 대규모 환경 관리를 자동화합니다.⁸

요약하자면, Azure Arc-enabled Kubernetes의 GitOps는 선언적 구성, 버전 제어, 자동화를 통해 하이브리드 및 멀티클라우드 환경에서 Kubernetes 클러스터를 안정적이고 안전하며 확장 가능한 방식으로 관리하기 위한 현대적인 DevOps 프랙티스입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-arc/kubernetes/conceptual-gitops-flux2>

문항 7:

Azure Monitor의 메트릭(Metrics)과 로그(Logs)의 차이점은 무엇이며, 각각 어떤 시나리오에 적합합니까?

Azure Monitor는 Azure 리소스의 성능과 상태를 파악하기 위한 두 가지 기본 데이터 유형인 **메트릭(Metrics)**과 **로그(Logs)**를 수집하고 분석합니다. 이 둘은 데이터의 구조, 수집 방식, 그리고 주요 사용 사례에서 뚜렷한 차이를 보이며, 효과적인 모니터링 전략을 수립하기 위해서는 이 차이점을 이해하는 것이 매우 중요합니다.¹²

특징	Azure Monitor 메트릭 (Metrics)	Azure Monitor 로그 (Logs)
데이터 유형	숫자 값 (Numerical values)	텍스트 기반 데이터 (Text-based data), 구조화 또는 비구조화
구조	시계열 데이터 (Time-series data). 타임스탬프, 이름, 값, 그리고 하나 이상의 차원(dimension)으로 구성됨.	레코드 집합. 각 레코드는 고유한 속성 집합을 가짐.
주요 사용 사례	성능 모니터링, 추세 분석, 실시간 경고	심층 진단, 근본 원인 분석(Root Cause Analysis), 감사, 복잡한 쿼리
쿼리 언어	차트 탐색기 UI 또는 필터 기반 API	Kusto 쿼리 언어 (KQL)
대기 시간 (Latency)	낮음 (거의 실시간, 일반적으로 1분). 사전 집계됨.	상대적으로 높음 (몇 분). 수집 및 인덱싱 시간 필요.
일반적인 시나리오	CPU 사용률, 네트워크 입/출력, 디스크 IOPS, 요청 수	애플리케이션 이벤트 로그, 웹 서버 요청 로그, 활동 로그, 보안 감사 로그

Azure Monitor 메트릭 (Metrics):

메트릭은 특정 시점의 시스템 상태를 나타내는 **가벼운 숫자 값**입니다. 이 데이터는 일정한 간격(보통 1분)으로 수집되며, 타임스탬프, 값, 그리고 컨텍스트를 제공하는 차원(dimensions)으로 구성됩니다.¹² 예를 들어, 가상 머신의 'Percentage CPU' 메트릭은 'InstanceName'이라는 차원을 가질 수 있어, 여러 VM 인스턴스 각각의 CPU 사용률을 개별적으로 추적할 수 있습니다.

메트릭 데이터는 수집 시점에 **사전 집계(pre-aggregated)**되므로 쿼리 성능이 매우 빠르고 대기 시간이 짧습니다. 이 특성 덕분에 메트릭은 다음과 같은 시나리오에 이상적입니다 ¹³:

- **실시간 성능 모니터링:** Azure Portal의 메트릭 탐색기를 사용하여 CPU 사용률, 응답 시간 등의 성능 지표를 시각적으로 차트화하고 추세를 분석합니다.
- **임계값 기반 경고:** 'CPU 사용률이 지난 5분 동안 90%를 초과하면 경고 발생'과 같이 간단하고 빠른 조건의 경고를 설정하는 데 적합합니다. 동적 임계값 기능을 사용하면 기계 학습을 통해 정상적인 동작 패턴을 학습하고 이탈을 감지할 수도 있습니다.¹⁴
- **자동 크기 조정(Autoscaling):** 가상 머신 확장 집합(VMSS)이나 App Service 플랜과 같은 서비스는 메트릭 값(예: CPU 부하, 큐 길이)에 따라 인스턴스 수를 자동으로 늘리거나 줄일 수 있습니다.

Azure Monitor 로그 (Logs):

로그는 애플리케이션, 운영 체제, Azure 서비스 등에서 생성된 **풍부한 이벤트 데이터**를 포함합니다. 각 로그 레코드는 다양한 속성을 가진 구조화된(또는 반구조화된) 데이터 집합입니다. 예를 들어, 웹 서버의 요청 로그는 타임스탬프, 클라이언트 IP, 요청 URL, 응답 코드, 응답 시간 등 다양한 정보를 포함할 수 있습니다.

로그 데이터는 Log Analytics 작업 영역에 수집되며, 강력한 **Kusto 쿼리 언어(KQL)**를 사용하여 분석됩니다. KQL을 사용하면 복잡한 조인, 집계, 통계 분석 등을 수행할 수 있습니다. 이러한 특성 때문에 로그는 다음과 같은 시나리오에 적합합니다 ¹⁴:

- **근본 원인 분석:** 특정 오류가 발생했을 때, 관련 로그를 쿼리하여 오류 메시지, 스택 추적, 그리고 오류 발생 전후의 이벤트 순서를 파악하여 문제의 원인을 심층적으로 진단합니다.
- **보안 감사 및 포렌식:** 활동 로그(Activity Logs)를 분석하여 '누가, 언제, 무엇을' 했는지 추적하거나, 보안 이벤트를 분석하여 잠재적인 위협을 탐지합니다.
- **복잡한 조건의 경고:** '지난 1시간 동안 특정 사용자가 5개 이상의 다른 IP 주소에서 로그인에 실패한 경우 경고'와 같이 여러 조건을 조합하거나 복잡한 논리가 필요한 경고를 설정합니다.

결론적으로, 메트릭은 시스템의 '건강 신호(health signals)'를 빠르고 가볍게 확인하는 데 사용되는 반면, 로그는 문제 발생 시 '부검(autopsy)'을 통해 상세한 원인을 파악하는 데 사용됩니다. 이 두 데이터 유형을 함께 사용하면 Azure 환경에 대한 포괄적인 가시성과 제어 능력을 확보할 수 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-monitor/data-platform>

문항 8:

Azure Monitor에서 메트릭 경고(Metric Alert)를 구성할 때 '차원(Dimensions)'을 사용하는 이유는 무엇이며, 구체적인 사용 사례는 무엇입니까?

Azure Monitor에서 메트릭 경고를 구성할 때 **차원(Dimensions)**을 사용하는 주된 이유는 단일 경고 규칙으로 여러 리소스 인스턴스나 세그먼트를 **개별적으로 모니터링**하기 위함입니다. 차원은 메트릭 값에 대한 추가적인 컨텍스트를 제공하는 이름/값 쌍으로, 데이터를 필터링하고 분할하는 데 사용됩니다.¹² 차원을 사용하지 않으면 여러 인스턴스의 메트릭이 집계되어 개별 인스턴스의 문제를 파악하기 어렵습니다.

예를 들어, 10개의 가상 머신(VM) 인스턴스로 구성된 가상 머신 확장 집합(VMESS)이 있다고 가정해 보겠습니다. 'Percentage CPU'라는 메트릭은 기본적으로 모든 인스턴스의 평균 CPU 사용률을 나타냅니다. 만약 단 하나의 VM 인스턴스만 CPU가 100%에 도달하고 나머지 9개는 유휴 상태라면, 평균 CPU 사용률은 10%에 불과하여 임계값 기반 경고가 트리거되지 않을 수 있습니다. 이로 인해 특정 인스턴스의 심각한 성능 저하를 놓치게 됩니다.

이때 '차원'이 해결책을 제공합니다. 'Percentage CPU' 메트릭은 `VMName` 또는 `InstanceId`와 같은 차원을 포함합니다. 경고 규칙을 구성할 때 이 차원을 기준으로 데이터를 분할하도록 설정하면, Azure Monitor는 각 VM 인스턴스를 별도의 시계열 데이터로 취급합니다. 따라서 단일 경고 규칙이 10개의 개별적인 모니터를 생성하는 효과를 냅니다. 이제 특정 VM 인스턴스 하나의 CPU 사용률이 임계값을 초과하면, 해당 인스턴스에 대해서만 경고가 발생합니다.¹⁴

차원 사용의 주요 이점:

1. **세분화된 모니터링:** 집계된 데이터 뒤에 숨겨진 개별 구성 요소의 문제를 식별할 수 있습니다.
2. **관리 효율성:** 수십, 수백 개의 리소스 인스턴스에 대해 각각 별도의 경고 규칙을 만들 필요 없이, 단일 규칙으로 전체 그룹을 관리할 수 있습니다. 이는 구성의 복잡성을 크게 줄여줍니다.
3. **동적 환경 지원:** 자동 크기 조정(Autoscaling)으로 인해 인스턴스가 동적으로 추가되거나 제거되는 환경에서도 효과적입니다. 새로운 인스턴스가 추가되면 경고 규칙이 자동으로 해당 인스턴스를 모니터링 대상에 포함합니다.

구체적인 사용 사례:

- **가상 머신 확장 집합(VMESS) 또는 다중 VM 워크로드:** `VMName` 차원을 사용하여 개별 VM 인스턴스의 CPU, 메모리, 디스크 사용량을 모니터링합니다.
- **스토리지 계정:** `ApiName` 차원을 사용하여 특정 스토리지 API 작업(예: `GetBlob`, `PutBlob`)의 대기 시간이나 성공/실패율을 모니터링합니다. 이를 통해 특정 API 호출에서 발생하는 성능 문제를 식별할 수 있습니다.
- **App Service:** `Instance` 차원을 사용하여 웹 앱을 실행하는 개별 서버 인스턴스의 요청 수나 응답 시간을 모니터링합니다.
- **Event Hubs/Service Bus:** `EntityName` 차원을 사용하여 단일 네임스페이스 내의 여러 이벤트 허브 또는 큐/토픽 각각의 들어오고 나가는 메시지 수를 모니터링합니다.
- **SQL Database:** `DatabaseName` 차원을 사용하여 단일 SQL 서버 내의 여러 데이터베이스 각각에 대한 DTU 사용률이나 교착 상태(deadlock) 수를 모니터링합니다.

결론적으로, 차원은 대규모 또는 다중 인스턴스 환경에서 메트릭 데이터를 의미 있게 분석하고 경고를 설정하기 위한 필수적인 기능입니다. 이를 통해 모니터링의 정확성과 효율성을 크게 향상시킬 수 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-monitor/alerts/alerts-metric-overview>

문항 9:

Azure Functions와 Azure Logic Apps는 모두 서버리스 워크플로우를 자동화하는 데 사용됩니다. 두 서비스의 주요 차이점은 무엇이며, 어떤 상황에서 하나를 다른 것보다 선호해야 할까요?

Azure Functions와 Azure Logic Apps는 모두 이벤트에 의해 트리거되는 서버리스 워크플로우를 구축하는 데 사용되는 강력한 서비스이지만, 설계 철학과 주요 사용 사례에서 근본적인 차이가 있습니다. 이 둘의 핵심적인 차이는 '코드 중심(Code-First)' 접근 방식과 '디자이너 중심(Designer-First)' 접근 방식에 있습니다.¹⁵

특징	Azure Functions	Azure Logic Apps
핵심 패러다임	코드 중심 (Code-First)	디자이너 중심 (Designer-First)
개발 방식	C#, F#, Java, Python, PowerShell, JavaScript 등의 코드로 비즈니스 로직을 작성. ¹⁶	시각적 디자이너에서 미리 만들어진 커넥터와 논리 블록을 연결하여 워크플로우를 구성. ¹⁷
주요 목적	이벤트에 대한 응답으로 특정 비즈니스 로직을 실행하는 서버리스 컴퓨팅 서비스.	다양한 서비스와 API를 통합하고 오케스트레이션하는 서버리스 워크플로우 서비스.
상태 관리	기본적으로 상태 비저장(Stateless)이지만, Durable Functions 확장을 통해 상태 저장(Stateful) 오케스트레이션을 코드로 구현 가능. ¹⁵	기본적으로 상태 저장(Stateful)이며, 워크플로우의 각 단계 실행 기록과 상태를 자동으로 추적.
커넥터	트리거(Triggers)와 바인딩(Bindings)을 통해 다른 서비스와 통합. 바인딩은 코드의 복잡성을 줄여주지만, Logic Apps만큼 다양하지는 않음.	수백 개의 관리형 커넥터(Managed Connectors) 라이브러리를 제공하여 코드 없이도 SaaS, PaaS, 온프레미스 시스템과 쉽게 통합 가능. ¹⁸
실행 환경	App Service 플랜, Premium 플랜, Consumption 플랜에서 실행 가능. 컨테이너나 Kubernetes에서도 실행할 수 있어 유연성이 높음.	다중 테넌트(Multi-tenant) 환경(Consumption) 또는 단일 테넌트(Single-tenant) 환경(Standard)에서 실행.
비용 모델	실행 횟수 및 리소스 사용량(실행 시간, 메모리)에 기반한 종량제.	실행 횟수 및 커넥터 작업 횟수에 기반한 종량제.

Azure Functions를 선호해야 하는 경우:

- **복잡한 비즈니스 로직 또는 데이터 처리:** 워크플로우에 표준 커넥터가 제공하지 않는 복잡한 계산, 데이터 변환, 또는 사용자 지정 알고리즘이 포함될 때 적합합니다. 예를 들어, 이미지 업로드 시 썸네일을 생성하거나, 복잡한 수학적 모델을 실행하는 경우입니다.
- **성능이 중요한 작업:** 짧은 대기 시간과 높은 처리량이 요구되는 작업에 더 적합합니다. 코드는 일반적으로 시각적 워크플로우 엔진보다 더 빠르게 실행될 수 있습니다.
- **기존 코드 라이브러리 활용:** 이미.NET, Java, Python 등으로 작성된 라이브러리나 종속성을 활용해야 할 때 Azure Functions는 자연스러운 선택입니다.

- **개발자 중심의 워크플로우:** 개발자가 코드 편집기(예: VS Code), 소스 제어(Git), CI/CD 파이프라인을 사용하여 완벽하게 제어하고 테스트하려는 경우에 이상적입니다.
- **상태 저장 오케스트레이션의 코드 제어:** Durable Functions를 사용하여 복잡한 워크플로우(예: 팬아웃/팬인, 승인 프로세스)를 코드로 직접 제어하고 싶을 때 사용합니다.

Azure Logic Apps를 선호해야 하는 경우:

- **신속한 통합 및 프로토타이핑:** 다양한 클라우드 서비스(예: Office 365, Salesforce, Azure Blob Storage)와 API를 신속하게 연결하고 통합해야 할 때 매우 강력합니다. 코드 작성 없이 시각적 디자이너를 통해 몇 분 만에 워크플로우를 만들 수 있습니다.¹⁷
- **B2B/EDI 시나리오:** EDIFACT, X12, AS2와 같은 표준 프로토콜을 지원하는 엔터프라이즈 통합 계정(Enterprise Integration Account) 기능을 통해 복잡한 B2B(Business-to-Business) 시나리오를 처리하는 데 특화되어 있습니다.¹⁹
- **시각적 워크플로우 및 비즈니스 프로세스 자동화:** '새로운 이메일이 도착하면 첨부 파일을 OneDrive에 저장하고 Teams에 알림 보내기'와 같이, 기술적 지식이 적은 사용자도 쉽게 이해하고 만들 수 있는 비즈니스 프로세스 자동화에 적합합니다.
- **견고한 상태 관리 및 재시도 정책:** 각 작업의 실행 상태, 입력, 출력이 자동으로 기록되므로 디버깅이 용이합니다. 또한, 일시적인 오류에 대비한 재시도 정책을 코드 없이 쉽게 구성할 수 있습니다.

함께 사용하는 시나리오:

두 서비스는 상호 배타적이지 않으며, 함께 사용할 때 더욱 강력한 솔루션을 만들 수 있습니다. Logic App 워크플로우 내에서 Azure Function을 하나의 작업(Action)으로 호출할 수 있습니다. 이는 'Best of Both Worlds' 접근 방식으로, Logic Apps의 강력한 통합 기능과 Functions의 강력한 컴퓨팅 능력을 결합하는 것입니다.

- **예시:** Logic App이 Salesforce에서 새로운 고객 레코드를 감지(트리거 및 커넥터)한 다음, Azure Function을 호출하여 해당 고객 데이터에 대한 복잡한 신용 점수 계산을 수행(코드 실행)하고, 그 결과를 다시 Logic App으로 받아 SQL 데이터베이스에 저장(커넥터)하는 워크플로우를 구성할 수 있습니다.

결론적으로, 선택은 워크플로우의 주요 목적에 따라 달라집니다. **통합과 오케스트레이션**이 중심이라면 Logic Apps로 시작하고, **사용자 지정 코드 실행과 데이터 처리**가 중심이라면 Functions로 시작하는 것이 좋습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-functions/functions-compare-logic-apps-ms-flow-webjobs>

문항 10:

Azure Functions의 호스팅 계획(Consumption, Premium, Dedicated)의 차이점을 설명하고, 각 계획이 적합한 시나리오를 제시하시오.

Azure Functions는 다양한 워크로드 요구사항과 예산에 맞춰 세 가지 주요 호스팅 계획을 제공합니다: **Consumption(소비) 계획**, **Premium(프리미엄) 계획**, 그리고 **Dedicated(App Service) 계획**. 각 계획은 크기 조정(scaling), 성능, 비용 및 네트워킹 기능 면에서 뚜렷한 차이점을 가집니다.¹⁶

1. Consumption Plan (소비 계획)

- **설명:** 가장 기본적인 서버리스 호스팅 모델입니다. Azure가 모든 인프라 관리를 처리하며, 함수가 실행될 때만 컴퓨팅 리소스에 대한 비용을 지불합니다. 이벤트가 발생하면 함수 앱이 신속하게 시작되고, 작업이 완료되면 리소스가 해제됩니다.

- **크기 조정:** 이벤트 수에 따라 자동으로 확장(scale out) 및 축소(scale in)됩니다. 트래픽이 급증하면 수백 개의 인스턴스로 빠르게 확장될 수 있습니다.
- **비용:** 실행 횟수(GB-seconds)와 총 실행 횟수에 따라 청구됩니다. 함수가 실행되지 않으면 비용이 발생하지 않으므로 비용 효율적입니다. 매월 무료 허용량이 제공됩니다.
- **제한 사항:**
 - 개별 함수 실행 시간은 기본적으로 5분, 최대 10분으로 제한됩니다.
 - '콜드 스타트(Cold Start)'가 발생할 수 있습니다. 즉, 한동안 호출되지 않은 함수는 첫 호출 시 인스턴스를 준비하는 데 약간의 지연 시간이 있을 수 있습니다.
 - VNet 통합과 같은 고급 네트워킹 기능이 제한됩니다.
- **적합한 시나리오:**
 - 트래픽이 간헐적이거나 예측 불가능한 이벤트 기반 워크로드 (예: 웹훅 처리, IoT 데이터 수신).
 - 단기간 실행되는 경량 작업.
 - 개발, 테스트 또는 비용에 민감한 애플리케이션.

2. Premium Plan (프리미엄 계획)

- **설명:** Consumption 계획의 자동 크기 조정 기능과 Dedicated 계획의 향상된 성능 및 네트워킹 기능을 결합한 모델입니다.
- **크기 조정:** Consumption 계획과 마찬가지로 이벤트에 따라 자동으로 확장됩니다. 하지만 핵심적인 차이점은 **미리 준비된 인스턴스(pre-warmed instances)**를 유지하여 콜드 스타트를 제거하거나 최소화할 수 있다는 것입니다.
- **비용:** 미리 준비된 인스턴스 수와 실행 중 사용된 리소스에 대해 시간당 비용이 발생합니다. Consumption 계획보다 비싸지만, 일관된 성능을 제공합니다.
- **주요 기능:**
 - **콜드 스타트 없음:** 항상 '웜(warm)' 상태인 인스턴스를 지정하여 요청에 즉시 응답할 수 있습니다.
 - **긴 실행 시간:** 함수 실행 시간이 최대 60분(기본 30분)까지 가능하여 장기 실행 작업에 적합합니다.
 - **VNet 통합:** 함수 앱을 가상 네트워크(VNet)에 연결하여 프라이빗 엔드포인트나 온프레미스 리소스에 안전하게 액세스할 수 있습니다.
 - 더 강력한 컴퓨팅 인스턴스(CPU 및 메모리)를 선택할 수 있습니다.
- **적합한 시나리오:**
 - 콜드 스타트로 인한 지연을 허용할 수 없는 프로덕션 워크로드.
 - 일관되게 높은 처리량이 필요한 스트리밍 데이터 처리.
 - VNet 내의 다른 Azure 서비스(예: SQL Database, Storage)에 비공개적으로 액세스해야 하는 함수.
 - 10분 이상 실행되어야 하는 데이터 처리 또는 오케스트레이션 작업.

3. Dedicated (App Service) Plan (전용 계획)

- **설명:** 함수 앱을 기존의 App Service 계획 내에서 실행합니다. 이는 웹 앱, API 앱, 모바일 앱과 동일한 리소스(VM 인스턴스)를 공유하는 것을 의미합니다.
- **크기 조정:** App Service 계획의 규칙에 따라 수동으로 또는 자동 크기 조정 규칙을 설정하여 크기를 조정합니다. 이벤트 기반의 동적 크기 조정은 제공되지 않으며, 항상 실행 중인 VM 인스턴스에 대해 비용을 지불합니다.
- **비용:** App Service 계획 자체에 대한 비용을 지불하므로, 함수 실행 여부와 관계없이 고정 비용이 발생합니다.

- **주요 기능:**

- 가장 예측 가능한 비용 모델을 제공합니다.
- App Service 계획이 제공하는 모든 기능(예: 배포 슬롯, Always On 설정)을 활용할 수 있습니다.
- 사용률이 낮은 기존 App Service 계획이 있는 경우, 추가 비용 없이 함수 앱을 호스팅하여 리소스를 효율적으로 사용할 수 있습니다.

- **적합한 시나리오:**

- 이미 사용 중인 App Service 계획의 남은 용량을 활용하고자 할 때.
- 트래픽이 지속적이고 예측 가능하여 고정된 수의 VM 인스턴스가 필요한 경우.
- 함수 앱을 다른 웹 애플리케이션과 동일한 환경 및 리소스에서 실행해야 하는 경우.
- 사용자 지정 이미지나 특정 OS 구성이 필요한 경우 (App Service Environment 사용 시).

요약:

선택은 애플리케이션의 성능 요구사항, 트래픽 패턴, 예산, 그리고 네트워킹 요구사항에 따라 결정됩니다. 비용 효율성과 간헐적 트래픽에는 Consumption, 성능 일관성과 VNet 연결에는 Premium, 기존 리소스 활용과 예측 가능한 비용에는 Dedicated 계획이 적합합니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-functions/functions-scale>

문항 11:

Durable Functions란 무엇이며, 일반적인 Azure Functions와 비교하여 어떤 종류의 문제를 해결합니까?

Durable Functions는 Azure Functions의 확장 기능으로, 서버리스 환경에서 **상태 저장(stateful) 워크플로우**를 코드로 작성할 수 있게 해줍니다. 일반적인 Azure Functions는 기본적으로 상태 비저장(stateless)이며, 짧은 시간 동안 실행되도록 설계되었습니다. 이로 인해 여러 함수 호출에 걸쳐 상태를 유지하거나 복잡한 순서의 작업을 오케스트레이션하는 것이 어렵습니다. Durable Functions는 이러한 한계를 극복하기 위해 등장했습니다.¹⁵

Durable Functions는 백그라운드에서 Azure Storage(큐, 테이블, 블롭)를 사용하여 함수 실행 상태, 체크포인트, 기록 등을 자동으로 관리합니다. 이를 통해 개발자는 인프라에 대한 걱정 없이 복잡한 상태 저장 로직을 코드에 집중하여 작성할 수 있습니다.

Durable Functions가 해결하는 문제:

일반적인 상태 비저장 함수로는 구현하기 복잡하거나 불가능한 다음과 같은 시나리오들을 간단하게 해결합니다.

1. 함수 체이닝 (Function Chaining):

- **문제:** 여러 함수를 특정 순서대로 실행해야 하는 경우. 예를 들어, 함수 A의 출력을 함수 B의 입력으로 사용하고, 함수 B의 출력을 함수 C의 입력으로 사용해야 하는 경우.
- **Durable Functions 해결책:** 오케스트레이터 함수 내에서 간단한 `await` 구문을 사용하여 함수들을 순차적으로 호출할 수 있습니다. Durable Functions 프레임워크가 중간 결과와 실행 상태를 자동으로 저장하고 전달해줍니다.

2. 팬아웃/팬인 (Fan-out/Fan-in):

- **문제:** 병렬로 여러 개의 독립적인 작업을 실행한 다음, 모든 작업이 완료될 때까지 기다렸다가 그 결과들을 집계해야 하는 경우. 예를 들어, 비디오 파일을 여러 세그먼트로 나누어 동시에 인코딩한 후, 모든 인코딩이 끝나면 하나의 파일로 합치는 작업.

- **Durable Functions 해결책:** 오케스트레이터 함수가 여러 개의 액티비티 함수를 동시에 호출(팬아웃)하고, `Task.WhenAll` 과 같은 구문을 사용하여 모든 작업이 완료되기를 기다린 후(팬인), 결과를 집계할 수 있습니다.

3. 비동기 HTTP API (Asynchronous HTTP APIs):

- **문제:** 완료하는 데 몇 분 또는 몇 시간이 걸리는 장기 실행 작업을 HTTP 요청으로 트리거해야 하는 경우, 클라이언트는 작업이 끝날 때까지 HTTP 연결을 유지할 수 없습니다.
- **Durable Functions 해결책:** HTTP 트리거 함수가 즉시 Durable 오케스트레이션을 시작하고, 클라이언트에게는 상태를 쿼리할 수 있는 URL(Status Query URL)이 포함된 `202 Accepted` 응답을 반환합니다. 클라이언트는 이 URL을 주기적으로 폴링하여 작업의 진행 상태(실행 중, 완료, 실패)를 확인하고 최종 결과를 얻을 수 있습니다.

4. 모니터링 (Monitoring):

- **문제:** 외부 이벤트나 사람의 개입을 기다리는 워크플로우를 구현해야 하는 경우. 예를 들어, 승인 프로세스에서 관리자가 승인 또는 거부할 때까지 워크플로우가 대기해야 합니다.
- **Durable Functions 해결책:** 오케스트레이션은 `WaitForExternalEvent` API를 사용하여 특정 이벤트가 발생할 때까지 실행을 '일시 중단'할 수 있습니다. 이 상태에서는 컴퓨팅 비용이 발생하지 않습니다. 외부 시스템이나 사용자가 이벤트를 보내면 오케스트레이션이 자동으로 재개됩니다.

5. 사람의 상호작용 (Human Interaction):

- **문제:** 승인 이메일을 보내고 사용자의 응답을 기다리는 것과 같이 사람의 개입이 필요한 워크플로우.
- **Durable Functions 해결책:** 모니터링 패턴과 유사하게, 오케스트레이션은 승인 요청을 보낸 후 외부 이벤트를 기다리며 대기할 수 있습니다. 타임아웃을 설정하여 일정 시간 동안 응답이 없으면 에스컬레이션 로직을 실행하는 등의 복잡한 시나리오도 쉽게 구현할 수 있습니다.

Durable Functions의 구성 요소:

Durable Functions는 주로 세 가지(또는 네 가지) 유형의 함수로 구성됩니다 15:

- **오케스트레이터 함수 (Orchestrator Function):** 워크플로우의 로직을 코드로 정의합니다. 액티비티 함수를 호출하고 실행 순서를 제어합니다. 결정론적(deterministic)이어야 한다는 제약 조건이 있습니다.
- **액티비티 함수 (Activity Function):** 실제 작업을 수행하는 함수입니다. 네트워크 호출, CPU 집약적 계산 등 비결정론적이고 장기 실행되는 작업을 처리합니다.
- **클라이언트 함수 (Client Function):** 오케스트레이션을 시작하는 진입점 역할을 하는 함수입니다. 일반적으로 HTTP 트리거, 큐 트리거 등이 클라이언트 함수가 됩니다.
- **엔티티 함수 (Entity Function):** 작고 상태 저장 가능한 '액터(actor)'를 통해 상태를 읽고 업데이트하는 데 사용됩니다. (고급 패턴)

결론적으로, Durable Functions는 서버리스 컴퓨팅의 단순성과 확장성을 유지하면서도, 복잡하고 상태 저장이 필요한 비즈니스 프로세스 오케스트레이션을 가능하게 하는 강력한 프로그래밍 모델입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/azure-functions/durable/durable-functions-overview>

문항 12:

Azure Logic Apps의 주요 구성 요소(워크플로우, 커넥터, 트리거, 작업)에 대해 설명하시오.

Azure Logic Apps는 시각적 디자이너를 사용하여 비즈니스 프로세스를 자동화하고 시스템을 통합하는 서비스입니다. Logic App의 핵심은 네 가지 주요 구성 요소인 **워크플로우**, **커넥터**, **트리거**, **작업**으로 이루어집니다.¹⁸

1. 워크플로우 (Workflow):

- 워크플로우는 Logic App의 가장 기본적인 단위로, 비즈니스 프로세스를 정의하는 일련의 단계(steps)입니다. "새로운 이메일이 오면, 첨부파일을 Blob Storage에 저장하고, Teams에 알림을 보낸다"와 같은 논리적 흐름 전체를 의미합니다.¹⁸
- 모든 워크플로우는 단 하나의 **트리거**로 시작하고, 그 뒤를 이어 하나 이상의 **작업**이 순차적으로 또는 조건부로 실행되는 구조를 가집니다.
- Azure Logic Apps는 이 워크플로우의 실행 상태, 입력, 출력을 모두 기록하여 모니터링과 디버깅을 용이하게 합니다.

2. 커넥터 (Connectors):

- 커넥터는 Logic Apps가 다른 서비스, API, 시스템, 프로토콜과 통신할 수 있도록 미리 만들어진 **프록시 또는 래퍼(wrapper)**입니다. 커넥터는 복잡한 API 호출, 인증, 데이터 형식 변환 등을 추상화하여 사용자가 코드 없이도 외부 시스템과 쉽게 통합할 수 있도록 돕습니다.¹⁷
- Azure는 수백 개의 커넥터를 제공하며, 이는 다음과 같이 분류될 수 있습니다:
 - **Azure 서비스 커넥터**: Azure Blob Storage, Service Bus, Azure Functions, Event Grid 등.
 - **SaaS/PaaS 커넥터**: Office 365 Outlook, Salesforce, Twitter, Dropbox, Dynamics 365 등.
 - **온프레미스 커넥터**: 온프레미스 데이터 게이트웨이를 통해 SQL Server, SharePoint Server, 파일 시스템 등과 연결합니다.
 - **프로토콜 커넥터**: HTTP, FTP, SMTP, SFTP 등 표준 프로토콜을 지원합니다.
- 각 커넥터는 특정 서비스에 대해 사용할 수 있는 트리거와 작업의 집합을 제공합니다.

3. 트리거 (Triggers):

- 트리거는 워크플로우를 **시작시키는 이벤트**입니다. 모든 Logic App 워크플로우는 반드시 하나의 트리거로 시작해야 합니다.¹⁷ 트리거가 발생(fire)하면 Logic Apps 엔진이 새로운 워크플로우 인스턴스를 생성하고 실행합니다.
- 트리거에는 여러 유형이 있습니다:
 - **폴링 트리거 (Polling Triggers)**: 주기적으로 서비스 엔드포인트를 확인하여 새로운 데이터가 있는지 검사합니다. 예를 들어, 'Office 365 Outlook' 커넥터의 '새 이메일이 도착할 때' 트리거는 1분마다 사서함을 확인합니다.
 - **푸시 트리거 (Push Triggers)**: 이벤트가 발생했을 때 서비스가 Logic App의 엔드포인트로 직접 데이터를 푸시(push)하기를 기다립니다. HTTP 요청 트리거가 대표적인 예입니다. 이 방식은 폴링보다 더 즉각적이고 효율적입니다.
 - **되풀이 트리거 (Recurrence Triggers)**: 특정 일정(예: 매일 오전 9시, 매시간)에 따라 워크플로우를 시작합니다.

4. 작업 (Actions):

- 작업은 트리거가 발생한 후 워크플로우에서 실행되는 **개별 단계**입니다. 각 작업은 특정 커넥터가 제공하는 함수나 오퍼레이션에 해당합니다. 예를 들어, 'Azure Blob Storage' 커넥터는 'Blob 생성', 'Blob 삭제', 'Blob 메타데이터 가져오기'와 같은 작업을 제공합니다.
- 작업은 이전 단계(트리거 또는 다른 작업)의 출력을 입력으로 사용할 수 있습니다. 이를 통해 데이터를 단계별로 전달하고 처리할 수 있습니다.
- 작업은 데이터를 조작하거나(예: JSON 구문 분석), 프로그램 흐름을 제어하는 제어 작업(예: 조건, For each 루프, Switch 문)일 수도 있습니다.

이 네 가지 구성 요소는 서로 유기적으로 결합하여 강력한 자동화 솔루션을 만듭니다. 예를 들어, **(트리거)** 'HTTP 요청을 수신하면' → **(작업 1)** '요청 본문의 JSON을 구문 분석하고' → **(작업 2)** 'Salesforce 커넥터를 사용하여 새 리드를 생성한 다음' → **(작업 3)** '조건에 따라 성공 또는 실패 이메일을 보낸다'와 같은 **(워크플로우)**를 구성할 수 있습니다. 이 모든 과정은 시각적 디자이너에서 **(커넥터)**를 조합하여 이루어집니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/logic-apps/logic-apps-overview>

문항 13:

Microsoft Entra ID(구 Azure Active Directory)란 무엇이며, 클라우드 환경에서 어떤 핵심적인 역할을 수행합니까?

Microsoft Entra ID(이전 명칭: Azure Active Directory, Azure AD)는 Microsoft의 클라우드 기반 **ID 및 액세스 관리 (Identity and Access Management, IAM) 서비스**입니다. 이는 조직의 직원, 파트너, 고객이 다양한 리소스에 안전하게 액세스할 수 있도록 하는 중앙 집중식 인증 및 권한 부여 시스템입니다.²⁰ Microsoft Entra ID는 단순히 기존의 온프레미스 Active Directory를 클라우드로 옮겨 놓은 것이 아니라, 최신 클라우드 및 SaaS 애플리케이션 환경에 맞게 설계된 포괄적인 ID 솔루션입니다.

Microsoft Entra ID의 핵심 역할:

1. 인증 (Authentication):

- 사용자가 누구인지 확인하는 프로세스입니다. Entra ID는 사용자 이름/암호, 다단계 인증(MFA), 암호 없는 인증 (Passwordless, 예: Microsoft Authenticator 앱, FIDO2 키), 생체 인식 등 다양한 최신 인증 방법을 지원합니다.
- 이를 통해 사용자는 단일 ID로 수천 개의 SaaS 애플리케이션(예: Microsoft 365, Salesforce, ServiceNow)과 Azure에 배포된 자체 개발 애플리케이션에 로그인할 수 있습니다.

2. 권한 부여 (Authorization):

- 인증된 사용자가 특정 리소스에 액세스할 수 있는지 여부를 결정하는 프로세스입니다. Entra ID는 이 역할을 주로 두 가지 강력한 기능을 통해 수행합니다.
- **Azure RBAC (역할 기반 액세스 제어):** Azure 리소스에 대한 세분화된 액세스 권한을 관리합니다.
- **조건부 액세스 (Conditional Access):** Entra ID의 핵심 기능으로, 'If-Then' 방식의 정책을 통해 액세스 제어를 자동화합니다. 예를 들어, "만약 사용자가 신뢰할 수 없는 위치에서 로그인하려고 하면, 반드시 MFA를 요구하라" 또는 "만약 디바이스가 규정을 준수하지 않으면, 민감한 데이터에 대한 액세스를 차단하라"와 같은 동적이고 상황에 맞는 정책을 적용할 수 있습니다.²⁰

3. Single Sign-On (SSO):

- 사용자가 한 번의 로그인으로 여러 애플리케이션과 서비스에 액세스할 수 있게 합니다. 이는 사용자 편의성을 크게 향상시키고, 여러 개의 암호를 관리해야 하는 부담을 줄여 보안을 강화합니다.

4. ID 거버넌스 (Identity Governance):

- 올바른 사람들이 올바른 리소스에 대한 올바른 액세스 권한을 갖도록 보장합니다. Entra ID는 액세스 검토(Access Reviews), 권한 관리(Entitlement Management), Privileged Identity Management(PIM)와 같은 기능을 통해 관리자 권한의 남용을 방지하고, 시간이 지남에 따라 누적된 불필요한 권한을 정기적으로 검토하고 제거하는 프로세스를 자동화합니다.

5. 하이브리드 ID 관리:

- 대부분의 기업은 온프레미스 Active Directory(AD)와 클라우드 환경을 함께 사용합니다. Microsoft Entra Connect 도구를 사용하면 온프레미스 AD의 사용자 계정을 Entra ID와 동기화하여 **하이브리드 ID**를 만들 수 있습니다. 이를 통해 사용자는 동일한 자격 증명으로 온프레미스와 클라우드 리소스 모두에 원활하게 액세스할 수 있습니다.²⁰

6. B2B 및 B2C ID 관리:

- **Azure AD B2B (Business-to-Business):** 외부 파트너나 공급업체 사용자를 게스트로 초대하여 조직의 애플리케이션과 리소스를 공유할 수 있도록 안전한 협업을 지원합니다.
- **Azure AD B2C (Business-to-Consumer):** 고객을 대상으로 하는 웹 및 모바일 애플리케이션을 위한 별도의 ID 관리 솔루션입니다. 고객이 소셜 계정(Google, Facebook 등)이나 로컬 계정으로 가입하고 로그인할 수 있도록 하며, 수백만 명의 사용자를 지원하도록 확장할 수 있습니다.²¹

결론적으로, Microsoft Entra ID는 현대적인 IT 환경의 '디지털 ID 제어 평면' 역할을 합니다. 단순히 사용자를 관리하는 디렉터리 서비스를 넘어, 제로 트러스트(Zero Trust) 보안 모델을 구현하는 핵심 요소로서, 모든 액세스 요청을 명시적으로 확인하고, 최소 권한 액세스를 강제하며, 잠재적 침해를 가정하는 보안 전략의 중심에 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/entra/fundamentals/whatis>

문항 14:

Azure의 관리 그룹(Management Groups), 구독(Subscriptions), 리소스 그룹(Resource Groups)의 계층 구조와 각각의 용도는 무엇입니까?

Azure는 리소스를 효율적으로 관리하고 거버넌스를 적용하기 위해 **관리 그룹(Management Groups), 구독(Subscriptions), 리소스 그룹(Resource Groups)**이라는 명확한 계층 구조를 제공합니다. 이 계층 구조는 정책 적용, 액세스 제어, 비용 관리를 위한 범위(Scope)를 정의하는 데 사용됩니다.¹

계층 구조:

가장 상위 수준부터 하위 수준까지의 구조는 다음과 같습니다.

```

1 Root Management Group (테넌트 당 하나)
2   └─ Management Group (e.g., Production, Development)
3     └─ Management Group (e.g., Core Services, Applications)
4       └─ Subscription (e.g., Prod Subscription, Dev Subscription)
5         └─ Resource Group (e.g., WebApp-RG, Database-RG)
6           └─ Resource (e.g., VM, Storage Account, VNet)
```

각 계층의 용도:

1. 관리 그룹 (Management Groups):

- **용도:** 여러 구독에 걸쳐 **액세스, 정책, 규정 준수를 통합 관리**하기 위한 최상위 컨테이너입니다. 조직의 구조(예: 부서별, 지역별, 환경별)에 맞춰 구독들을 그룹화할 수 있습니다.¹
- **주요 기능:**
 - **거버넌스 상속:** 관리 그룹에 Azure Policy나 RBAC 역할을 할당하면, 해당 관리 그룹에 속한 모든 하위 관리 그룹과 구독에 해당 설정이 자동으로 상속됩니다. 이를 통해 조직 전체에 일관된 거버넌스 가이드라인을 설정할 수 있습니다.

- **대규모 정책 적용:** 예를 들어, '모든 프로덕션 환경에서는 특정 고비용 VM SKU 사용 금지'라는 정책을 'Production' 관리 그룹에 적용하면, 해당 그룹 내의 모든 프로덕션 관련 구독에 이 규칙이 강제됩니다.
- **통합 비용 보기:** Azure Cost Management에서 관리 그룹 수준의 집계된 비용을 확인하여 조직 전체의 클라우드 지출을 분석할 수 있습니다.

2. 구독 (Subscriptions):

- **용도:** Azure 리소스를 프로비저닝하기 위한 **관리, 청구, 및 스케일 단위**입니다. 모든 Azure 리소스는 반드시 하나의 구독에 속해야 합니다.²
- **주요 기능:**
 - **청구 단위:** Azure 사용량에 대한 비용은 구독 단위로 청구됩니다. 따라서 부서별 또는 프로젝트별로 구독을 분리하면 비용을 명확하게 추적하고 할당할 수 있습니다.
 - **규모 제한 경계:** Azure는 구독별로 리소스 수(예: 구독당 가상 네트워크 수)나 API 호출 속도 등에 대한 제한(Quotas and Limits)을 가지고 있습니다. 대규모 배포 시에는 이러한 제한을 고려하여 구독을 설계해야 합니다.
 - **관리 경계:** 구독은 관리의 경계 역할을 합니다. 예를 들어, 프로덕션 환경과 개발 환경을 별도의 구독으로 분리하여 개발자가 실수로 프로덕션 리소스를 변경하는 것을 방지할 수 있습니다.

3. 리소스 그룹 (Resource Groups):

- **용도:** 동일한 수명 주기를 공유하는 Azure 리소스를 담은 **논리적 컨테이너**입니다. 리소스 그룹은 배포 및 관리의 기본 단위 역할을 합니다.²
- **주요 기능:**
 - **그룹 관리:** 리소스 그룹 내의 모든 리소스는 하나의 단위로 관리될 수 있습니다. 예를 들어, 특정 애플리케이션을 구성하는 모든 리소스(웹 앱, 데이터베이스, 스토리지)를 하나의 리소스 그룹에 배치하면, 해당 애플리케이션 전체를 함께 배포, 업데이트, 모니터링, 삭제할 수 있습니다.
 - **수명 주기 관리:** 리소스 그룹을 삭제하면 그 안에 포함된 모든 리소스도 함께 삭제됩니다. 이는 프로젝트나 임시 환경을 정리할 때 매우 유용합니다.
 - **세분화된 액세스 제어:** 리소스 그룹 수준에서 RBAC 역할을 할당하여 특정 팀이나 사용자에게 해당 그룹 내의 리소스에 대한 액세스 권한만 부여할 수 있습니다.
 - **비용 추적:** 리소스 그룹별로 비용을 필터링하여 특정 애플리케이션이나 서비스의 운영 비용을 분석할 수 있습니다.

이 계층 구조를 전략적으로 활용하면 조직의 규모가 커지고 클라우드 사용량이 증가하더라도 체계적이고 안전하며 비용 효율적인 방식으로 Azure 환경을 관리할 수 있습니다. 거버넌스는 최상위(관리 그룹)에서 광범위하게 설정하고, 아래로 내려갈수록 더 세분화된 제어를 적용하는 것이 일반적인 모범 사례입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/governance/management-groups/overview>

문항 15:

Azure Blob Storage의 세 가지 액세스 계층(핫, 쿨, 아카이브)은 각각 어떤 특징을 가지며, 비용 최적화를 위해 이들을 어떻게 활용할 수 있습니까?

Azure Blob Storage는 데이터의 액세스 빈도와 보존 기간에 따라 비용을 최적화할 수 있도록 **핫(Hot)**, **쿨(Cool)**, **아카이브(Archive)**의 세 가지 온라인 액세스 계층을 제공합니다. 각 계층은 스토리지 비용과 액세스 비용 간의 트레이드오프를 기반으로 설계되었습니다.²²

각 액세스 계층의 특징:

1. 핫 (Hot) 계층:

- **특징:** 자주 액세스(읽기 및 쓰기)되는 데이터를 저장하기 위해 최적화되었습니다.
- **비용 구조:** 세 계층 중 스토리지 비용(**GB당 월 비용**)이 가장 높고, 액세스 비용(읽기/쓰기 작업 비용)이 가장 낮습니다.
- **대기 시간:** 밀리초 단위의 매우 낮은 액세스 대기 시간을 제공합니다.
- **사용 사례:**
 - 활발하게 사용되는 애플리케이션 데이터.
 - 웹사이트의 이미지나 비디오와 같이 직접 사용자에게 제공되는 콘텐츠.
 - 데이터 처리 파이프라인의 스테이징 데이터.

2. 쿨 (Cool) 계층:

- **특징:** 자주 액세스하지 않지만 필요할 때 즉시 사용할 수 있어야 하는 데이터를 저장하기 위해 최적화되었습니다. 데이터는 최소 30일 동안 보관해야 합니다.
- **비용 구조:** 핫 계층보다 스토리지 비용이 낮지만, 액세스 비용이 더 높습니다.
- **대기 시간:** 핫 계층과 동일하게 밀리초 단위의 대기 시간을 제공합니다.
- **사용 사례:**
 - 단기 백업 및 재해 복구 데이터.
 - 오래된 미디어 콘텐츠 또는 과학 데이터.
 - 규정 준수를 위한 감사 로그.

3. 아카이브 (Archive) 계층:

- **특징:** 거의 액세스하지 않는 장기 보관 데이터를 위해 설계되었습니다. 데이터는 최소 180일 동안 보관해야 합니다. 이 계층은 오프라인 상태로 간주됩니다.
- **비용 구조:** 세 계층 중 스토리지 비용이 압도적으로 가장 저렴하지만, 데이터 검색(retrieval) 비용과 시간이 가장 많이 듭니다.
- **대기 시간:** 데이터를 읽으려면 먼저 '리하이드레이션(rehydration)' 과정을 거쳐 핫 또는 쿨 계층으로 변경해야 합니다. 이 과정은 데이터 양과 검색 우선순위(표준 또는 높음)에 따라 몇 시간이 걸릴 수 있습니다.²²
- **사용 사례:**
 - 장기 데이터 보관 및 규정 준수 아카이브 (예: 의료 기록, 금융 거래 데이터).
 - 원본 데이터가 삭제된 후의 원시 데이터 백업.
 - 테이프 백업 대체.

액세스 계층을 활용한 비용 최적화: 수명 주기 관리 (Lifecycle Management)

수동으로 Blob의 계층을 변경하는 것은 비효율적이므로, Azure는 **수명 주기 관리 정책**이라는 강력한 자동화 도구를 제공합니다. 이 정책을 사용하면 규칙 기반으로 Blob을 계층 간에 자동으로 이동시키거나 삭제할 수 있습니다.²²

수명 주기 관리 정책 설정 예시:

- **규칙 1 (쿨 계층으로 이동):** 마지막으로 수정한 후 30일이 지난 Blob을 핫 계층에서 쿨 계층으로 이동합니다.
 - **효과:** 활성 데이터가 비활성 상태가 되면 더 저렴한 스토리지 비용의 쿨 계층으로 자동 전환됩니다.
- **규칙 2 (아카이브 계층으로 이동):** 마지막으로 수정한 후 90일이 지난 Blob을 쿨 계층에서 아카이브 계층으로 이동합니다.
 - **효과:** 단기 백업 데이터가 장기 보관용으로 전환되면 가장 저렴한 아카이브 계층으로 이동하여 스토리지 비용을 극대화합니다.
- **규칙 3 (삭제):** 마지막으로 수정한 후 7년(2555일)이 지난 Blob을 삭제합니다.
 - **효과:** 보존 기간이 만료된 데이터를 자동으로 삭제하여 불필요한 스토리지 비용 발생을 방지합니다.

이러한 정책은 스토리지 계정 수준에서 정의되며, 접두사(prefix)를 사용하여 특정 컨테이너나 폴더 구조에만 규칙을 적용할 수 있습니다.

결론적으로, Azure Blob Storage의 액세스 계층과 수명 주기 관리 정책을 전략적으로 사용하면 데이터의 가치와 액세스 패턴에 맞춰 스토리지 비용을 지능적으로 관리할 수 있습니다. 이는 대량의 비정형 데이터를 처리하는 모든 조직에게 필수적인 비용 최적화 기법입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/storage/blobs/storage-blob-storage-tiers>

문항 16:

Azure Storage 계정의 유형(Standard general-purpose v2, Premium block blobs 등)에는 어떤 것들이 있으며, 각각의 주요 사용 사례는 무엇입니까?

Azure Storage는 다양한 성능, 비용, 기능 요구사항을 충족시키기 위해 여러 유형의 스토리지 계정을 제공합니다. 애플리케이션의 워크로드에 맞는 올바른 계정 유형을 선택하는 것은 성능과 비용 효율성을 극대화하는 데 매우 중요합니다. 주요 스토리지 계정 유형은 다음과 같습니다.²³

1. Standard general-purpose v2 (표준 범용 v2)

- **지원 서비스:** Blob Storage (Block, Append, Page Blobs), Queue Storage, Table Storage, Azure Files. Data Lake Storage Gen2도 지원합니다.
- **성능:** 표준 HDD(하드 디스크 드라이브) 기반의 성능을 제공합니다.
- **중복성 옵션:** LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS 등 모든 중복성 옵션을 지원하여 유연성이 가장 높습니다.
- **주요 사용 사례:**
 - 대부분의 시나리오에 권장되는 기본 계정 유형입니다.
 - Blob, 파일 공유, 큐, 테이블 등 다양한 스토리지 서비스를 하나의 계정에서 사용해야 하는 경우.
 - 웹사이트 콘텐츠, 백업 및 아카이브, 빅데이터 분석용 데이터 저장 등 범용적인 목적에 적합합니다.

2. Premium block blobs (프리미엄 블록 Blob)

- **지원 서비스:** Block Blobs 및 Append Blobs만 지원합니다.
- **성능:** 고성능 SSD(솔리드 스테이트 드라이브)를 사용하여 낮은 대기 시간(low latency)과 높은 트랜잭션 속도를 제공하도록 최적화되었습니다.
- **중복성 옵션:** LRS, ZRS를 지원합니다.

- **주요 사용 사례:**

- 트랜잭션 속도가 매우 높거나, 작은 객체를 많이 처리하거나, 일관되게 매우 낮은 스토리지 대기 시간이 요구되는 시나리오.
- 대화형 비디오 편집, 고성능 컴퓨팅(HPC), AI/ML 모델 학습 데이터셋 등 빠른 데이터 접근이 필수적인 워크로드.

3. Premium file shares (프리미엄 파일 공유)

- **지원 서비스:** Azure Files만 지원합니다.

- **성능:** 고성능 SSD를 사용하여 엔터프라이즈급 파일 공유 워크로드에 최적화된 높은 IOPS와 처리량을 제공합니다.

- **중복성 옵션:** LRS, ZRS를 지원합니다.

- **주요 사용 사례:**

- 고성능을 요구하는 엔터프라이즈 파일 공유 애플리케이션.
- 데이터베이스(예: SQL Server), VDI(가상 데스크톱 인프라) 프로파일, 개발/테스트 환경 등 까다로운 워크로드.
- SMB(Server Message Block) 프로토콜과 NFS(Network File System) 프로토콜을 모두 지원하는 파일 공유가 필요한 경우.

4. Premium page blobs (프리미엄 페이지 Blob)

- **지원 서비스:** Page Blobs만 지원합니다.

- **성능:** 고성능 SSD 기반이며, 512바이트 페이지에 대한 빈번한 임의 읽기/쓰기 작업에 최적화되어 있습니다.

- **중복성 옵션:** LRS, ZRS를 지원합니다.

- **주요 사용 사례:**

- Azure 가상 머신(VM)의 OS 및 데이터 디스크로 사용되는 **Azure Managed Disks**의 기반 기술입니다. 일반적으로 사용자가 직접 이 계정 유형을 프로비저닝하기보다는 Azure VM을 생성할 때 간접적으로 사용하게 됩니다.
- 자주 액세스하는 파일에 대한 임의 읽기/쓰기 패턴을 가진 애플리케이션.

레거시 계정 유형:

- **Standard general-purpose v1 (표준 범용 v1)** 및 **Blob Storage (레거시)** 계정도 존재하지만, Microsoft는 최신 기능과 더 낮은 기가바이트당 가격을 제공하는 **Standard general-purpose v2** 계정을 사용할 것을 강력히 권장합니다.23 특별한 이유가 없는 한 새로운 배포에는 v2 계정을 사용해야 합니다.

선택 가이드:

- 범용성 및 비용 효율성이 중요하다면 **Standard general-purpose v2**를 선택합니다.
- 최고의 **Blob** 성능과 낮은 대기 시간이 필요하다면 **Premium block blobs**를 선택합니다.
- 고성능 파일 공유가 필요하다면 **Premium file shares**를 선택합니다.

각 계정 유형은 특정 워크로드에 맞춰진 성능 및 비용 특성을 가지므로, 애플리케이션의 요구 사항을 정확히 파악하고 가장 적합한 계정을 선택하는 것이 중요합니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/storage/common/storage-account-overview>

문항 17:

Azure Storage의 데이터 중복성 옵션(LRS, ZRS, GRS, GZRS)을 비교하고, 각 옵션의 가용성 SLA와 적합한 시나리오를 설명하시오.

Azure Storage는 하드웨어 장애, 네트워크 또는 전원 중단, 심지어 전체 데이터 센터의 재해로부터 데이터를 보호하기 위해 여러 데이터 중복성 옵션을 제공합니다. 각 옵션은 내구성(durability), 가용성(availability), 비용 간에 서로 다른 균형을 제공합니다.²³

1. 로컬 중복 스토리지 (LRS - Locally-redundant storage)

- **작동 방식:** 단일 Azure 지역 내의 **단일 데이터 센터**에서 데이터를 동기적으로 세 번 복제합니다.
- **보호 대상:** 서버 랙 및 드라이브 장애로부터 보호합니다.
- **보호 불가 대상:** 데이터 센터 전체에 영향을 미치는 재해(예: 화재, 홍수)로부터는 보호하지 못합니다.
- **가용성:** 최소 99.9% (읽기 및 쓰기).
- **비용:** 가장 저렴한 옵션입니다.
- **적합한 시나리오:**
 - 비용에 민감하고 데이터 손실 시 쉽게 재구성할 수 있는 비핵심 데이터.
 - 데이터 거버넌스 요구 사항으로 인해 데이터가 특정 국가/지역을 벗어나면 안 되는 경우.

2. 영역 중복 스토리지 (ZRS - Zone-redundant storage)

- **작동 방식:** 단일 Azure 지역 내의 **세 개의 서로 다른 가용성 영역(Availability Zones)**에 걸쳐 데이터를 동기적으로 세 번 복제합니다. 각 가용성 영역은 독립적인 전원, 냉각, 네트워킹을 갖춘 물리적으로 분리된 데이터 센터입니다.
- **보호 대상:** LRS가 보호하는 모든 장애와 더불어, 데이터 센터 수준의 장애로부터 보호합니다.
- **보호 불가 대상:** 지역 전체에 영향을 미치는 대규모 재해로부터는 보호하지 못합니다.
- **가용성:** 최소 99.99% (읽기 및 쓰기).
- **비용:** LRS보다 비싸지만 GRS보다 저렴합니다.
- **적합한 시나리오:**
 - 고가용성이 요구되는 애플리케이션.
 - 데이터를 특정 지역 내에 유지해야 하지만, 데이터 센터 수준의 장애에 대한 복원력이 필요한 경우.

3. 지역 중복 스토리지 (GRS - Geo-redundant storage)

- **작동 방식:** 두 단계의 중복성을 제공합니다.
 1. **LRS:** 주 지역(primary region)의 단일 데이터 센터 내에 데이터를 동기적으로 세 번 복제합니다.
 2. **비동기 복제:** 주 지역의 데이터를 수백 마일 떨어진 쌍을 이루는 보조 지역(secondary region)으로 비동기적으로 복제합니다. 보조 지역에서는 LRS를 사용하여 데이터가 다시 세 번 복제됩니다.
- **보호 대상:** LRS가 보호하는 모든 장애와 더불어, 지역 전체에 영향을 미치는 재해로부터 보호합니다.
- **보호 불가 대상:** 없음 (최고 수준의 내구성).
- **가용성:**
 - 주 지역: 최소 99.9% (읽기 및 쓰기).
 - 보조 지역: 주 지역 장애 조치(failover) 시 사용 가능. 읽기 액세스 GRS(RA-GRS)를 사용하면 평상시에도 보조 지역에서 읽기 액세스가 가능하며, 이 경우 읽기 가용성은 최소 99.99%입니다.

- **비용:** ZRS보다 비쌉니다.
- **적합한 시나리오:**
 - 지역적 재해에 대비한 최고 수준의 데이터 내구성이 필요한 미션 크리티컬 데이터.
 - 국가/지역 수준의 재해 복구(DR) 전략이 필요한 경우.

4. 지역-영역 중복 스토리지 (GZRS - Geo-zone-redundant storage)

- **작동 방식:** ZRS와 GRS의 장점을 결합한 최상위 옵션입니다.
 1. **ZRS:** 주 지역의 세 가용성 영역에 걸쳐 데이터를 동기적으로 복제합니다.
 2. **비동기 복제:** 주 지역의 데이터를 보조 지역으로 비동기적으로 복제하며, 보조 지역에서는 LRS를 사용하여 데이터가 보호됩니다.
- **보호 대상:** 데이터 센터 수준의 장애와 지역 수준의 재해 모두로부터 보호합니다.
- **가용성:**
 - 주 지역: 최소 99.99% (읽기 및 쓰기).
 - 보조 지역: 읽기 액세스 GZRS(RA-GZRS)를 사용하면 평상시에도 보조 지역에서 읽기 액세스가 가능하며, 이 경우 읽기 가용성은 최소 99.9999%입니다.
- **비용:** 가장 비싼 옵션입니다.
- **적합한 시나리오:**
 - 최고 수준의 가용성과 내구성이 동시에 요구되는 가장 중요한 워크로드.
 - 데이터 센터 장애 시에도 다운타임을 최소화해야 하고, 동시에 지역적 재해에 대한 보호도 필요한 경우.

요약 표:

중복성 옵션	복제 방식	보호 대상	주 지역 가용성 (읽기/쓰기)	비용
LRS	단일 데이터 센터, 3개 복사본	랙/드라이브 장애	≥ 99.9%	가장 낮음
ZRS	3개 가용성 영역, 3개 복사본	데이터 센터 장애	≥ 99.99%	낮음
GRS	주 지역(LRS) + 보조 지역(LRS)	지역 재해	≥ 99.9%	높음
GZRS	주 지역(ZRS) + 보조 지역(LRS)	데이터 센터 장애 + 지역 재해	≥ 99.99%	가장 높음

올바른 중복성 옵션을 선택하는 것은 애플리케이션의 가용성 요구사항, 데이터의 중요성, 재해 복구 목표, 그리고 예산을 종합적으로 고려해야 하는 중요한 아키텍처 결정입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/storage/common/storage-redundancy>

문항 18:

Azure Files와 Azure Blob Storage의 근본적인 차이점은 무엇이며, 각각 어떤 용도로 사용해야 하나요?

Azure Files와 Azure Blob Storage는 모두 Azure Storage 플랫폼의 일부이지만, 근본적으로 다른 목적을 위해 설계되었으며 서로 다른 액세스 프로토콜을 사용합니다. 이 둘의 핵심 차이는 **파일 시스템 공유**와 **객체 스토리지**라는 개념에 있습니다.²⁴

Azure Files:

- **핵심 개념:** 클라우드에서 완벽하게 관리되는 **서버리스 파일 공유(Serverless File Shares)**를 제공합니다.
- **액세스 프로토콜:** 업계 표준인 **SMB(Server Message Block)** 및 **NFS(Network File System)** 프로토콜을 사용하여 액세스합니다.
- **주요 특징:**
 - **익숙한 파일 시스템 경험:** Windows의 파일 탐색기나 Linux의 `mount` 명령어를 사용하여 로컬 드라이브처럼 탑재(mount)할 수 있습니다. 기존 애플리케이션을 코드 변경 없이 클라우드로 '리프트 앤 시프트(lift-and-shift)'하는 데 이상적입니다.
 - **동시 액세스:** 여러 가상 머신(VM)이나 온프레미스 서버에서 동시에 동일한 파일 공유를 탑재하고 파일을 읽고 쓸 수 있습니다.
 - **ID 기반 인증:** 온프레미스 Active Directory Domain Services(AD DS) 또는 Microsoft Entra Domain Services와 통합하여 익숙한 Windows ACL(액세스 제어 목록) 기반의 파일 및 폴더 수준 권한을 적용할 수 있습니다.²⁴
- **사용 사례:**
 - 온프레미스 파일 서버를 클라우드로 마이그레이션하거나 대체.
 - 여러 VM 간에 구성 파일, 로그 파일, 애플리케이션 데이터 등을 공유.
 - VDI(가상 데스크톱 인프라) 환경의 사용자 프로필 저장.
 - 개발 및 테스트 환경에서 공유 코드 리포지토리 또는 도구 저장.
 - Azure File Sync를 사용하여 온프레미스 Windows Server와 클라우드 파일 공유 간에 데이터를 동기화하여 하이브리드 시나리오 구현.²⁵

Azure Blob Storage:

- **핵심 개념:** 텍스트나 이진 데이터와 같은 **비정형 데이터(Unstructured Data)**를 대규모로 저장하기 위한 **객체 스토리지(Object Storage)**입니다.
- **액세스 프로토콜:** **HTTP/HTTPS**를 통한 REST API를 사용하여 액세스합니다. SDK(.NET, Java, Python 등), PowerShell, Azure CLI를 통해서도 상호작용합니다.²⁴
- **주요 특징:**
 - **엄청난 확장성:** 페타바이트(PB) 규모의 데이터를 저장하고 초당 수백만 건의 요청을 처리할 수 있도록 설계되었습니다.
 - **계층적 네임스페이스(Hierarchical Namespace) 미지원(기본적으로):** 전통적인 파일 시스템과 달리, Blob Storage는 기본적으로 플랫(flat)한 구조를 가집니다. 컨테이너 내에 Blob이 존재하며, 폴더 구조는 경로 이름의 일부로 시뮬레이션됩니다. (단, Data Lake Storage Gen2는 이 위에 계층적 네임스페이스를 제공합니다.)
 - **비용 효율적인 계층:** 핫, 쿨, 아카이브 액세스 계층을 제공하여 데이터 액세스 빈도에 따라 비용을 최적화할 수 있습니다.
- **사용 사례:**
 - 이미지, 비디오, 문서 등 웹 콘텐츠를 저장하고 브라우저에 직접 제공.

- 대규모 데이터 백업, 복원, 아카이빙.
- 빅데이터 분석(예: Azure Databricks, HDInsight)을 위한 데이터 레이크(Data Lake) 구축.
- 로그 파일, 원격 분석 데이터 등 스트리밍 데이터 저장.
- 정적 웹사이트 호스팅.²²

요약 비교:

특징	Azure Files	Azure Blob Storage
패러다임	관리형 파일 공유	대규모 객체 스토어
액세스 프로토콜	SMB, NFS	HTTP/HTTPS (REST API)
주요 용도	공유 스토리지, 리프트 앤 시프트	비정형 데이터, 데이터 레이크, 백업
액세스 방식	파일 시스템처럼 마운트	URL을 통해 객체에 접근
동시성	여러 클라이언트의 동시 읽기/쓰기 지원	대규모 병렬 읽기에 최적화됨
인증	AD DS/Entra ID 기반 파일/폴더 권한	Entra ID, SAS 토큰, 액세스 키

선택은 애플리케이션이 데이터를 어떻게 사용하고 액세스하는지에 따라 결정됩니다. 기존 파일 시스템 API를 사용하는 애플리케이션이나 여러 서버 간의 공유 스토리지가 필요하다면 **Azure Files**가 적합합니다. 웹을 통해 대규모 비정형 데이터에 접근하거나 데이터 레이크를 구축해야 한다면 **Azure Blob Storage**가 올바른 선택입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/storage/blobs/storage-blobs-introduction> / <https://learn.microsoft.com/ko-kr/azure/storage/files/storage-files-introduction>

문항 19:

Azure File Sync란 무엇이며, 하이브리드 클라우드 환경에서 어떤 문제를 해결해 줍니까?

Azure File Sync는 온프레미스 또는 클라우드에 있는 Windows Server를 Azure Files의 **빠른 캐시(fast cache)**로 변환하여, 중앙 집중식 파일 공유와 분산된 서버 간의 데이터 동기화를 자동화하는 서비스입니다. 이를 통해 조직은 데이터의 중앙 관리, 다중 사이트 액세스, 클라우드 계층화와 같은 하이브리드 클라우드 스토리지 시나리오를 효과적으로 구현할 수 있습니다.²⁵

Azure File Sync의 주요 구성 요소:

- **Azure 파일 공유:** 클라우드에 있는 중앙 집중식 파일 공유로, 모든 데이터의 마스터 복사본 역할을 합니다.
- **스토리지 동기화 서비스:** Azure에 배포되는 최상위 리소스로, Azure File Sync의 동기화 관계를 관리합니다.
- **동기화 그룹:** 동기화할 파일들의 토폴로지(topology)를 정의합니다. 동기화 그룹에는 하나의 **클라우드 엔드포인트**(Azure 파일 공유)와 하나 이상의 **서버 엔드포인트**가 포함됩니다.
- **서버 엔드포인트:** 동기화 그룹에 추가된 온프레미스 Windows Server의 특정 경로(예: `D:\Data`)입니다.
- **Azure File Sync 에이전트:** 온프레미스 Windows Server에 설치되어 Azure 파일 공유와의 동기화를 담당하는 에이전트입니다.

Azure File Sync가 해결하는 문제와 제공하는 이점:

1. 데이터 중앙 집중화 및 다중 사이트 동기화:

- **문제:** 여러 지사나 사무실을 가진 조직은 각 위치의 파일 서버 간에 데이터를 일관되게 유지하는 데 어려움을 겪습니다. 기존의 DFS-R(분산 파일 시스템 복제)과 같은 기술은 설정이 복잡하고 충돌 해결이 어려울 수 있습니다.
- **해결책:** Azure File Sync는 Azure 파일 공유를 모든 데이터의 단일 진실 공급원(single source of truth)으로 사용합니다. 각 지사의 Windows Server는 이 중앙 허브와 동기화됩니다. 한 서버에서 파일이 변경되면, 변경 사항이 먼저 Azure 파일 공유에 업로드된 다음 다른 모든 서버로 전파됩니다. 이를 통해 모든 위치에서 데이터 일관성을 쉽게 유지할 수 있습니다.

2. 온프레미스 스토리지 용량 절감 (클라우드 계층화):

- **문제:** 온프레미스 파일 서버의 스토리지 용량은 제한적이며, 데이터가 증가함에 따라 디스크를 계속 증설해야 하는 부담이 있습니다.
- **해결책:** Azure File Sync의 핵심 기능인 **클라우드 계층화(Cloud Tiering)**를 사용하면, 자주 액세스하지 않는 파일을 온프레미스 서버에서 Azure 파일 공유로 이동시키고, 서버에는 해당 파일에 대한 포인터(재분석 지점)만 남겨둘 수 있습니다. 사용자가 계층화된 파일을 열려고 하면, File Sync 에이전트가 Azure에서 파일을 원활하게 다운로드하여 제공합니다. 이를 통해 온프레미스 서버에서는 최소한의 로컬 캐시만 유지하면서, 클라우드의 거의 무한한 스토리지 용량을 활용할 수 있습니다. 관리자는 볼륨 여유 공간 정책(예: '항상 디스크의 20%를 비워두기')을 설정하여 이 프로세스를 자동화할 수 있습니다.

3. 빠른 파일 액세스 및 로컬 성능:

- **문제:** 모든 데이터를 클라우드에만 저장하면 원격지 사용자는 인터넷 대역폭과 대기 시간으로 인해 파일 액세스가 느릴 수 있습니다.
- **해결책:** Azure File Sync는 각 위치에 로컬 캐시를 제공합니다. 자주 사용하는 '핫(hot)' 파일은 로컬 서버에 캐시되므로, 사용자는 LAN 속도로 파일에 액세스할 수 있습니다. 클라우드 계층화를 사용하더라도, 파일은 로컬 서버를 통해 제공되므로 성능이 뛰어납니다.

4. 간편한 백업 및 재해 복구:

- **문제:** 온프레미스 파일 서버를 백업하는 것은 복잡하고 시간이 많이 소요될 수 있습니다.
- **해결책:** 모든 데이터의 마스터 복사본이 Azure 파일 공유에 있으므로, Azure Backup과 같은 클라우드 기반 백업 솔루션을 사용하여 중앙에서 간편하게 백업하고 스냅샷을 관리할 수 있습니다. 온프레미스 서버에 장애가 발생하더라도, 새로운 서버를 신속하게 설정하고 Azure File Sync를 통해 데이터를 빠르게 복원할 수 있습니다.

결론적으로, Azure File Sync는 기존의 온프레미스 파일 서버 투자를 활용하면서 클라우드의 확장성, 중앙 관리, 재해 복구 이점을 결합하고자 하는 조직에게 이상적인 하이브리드 스토리지 솔루션입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/storage/file-sync/file-sync-introduction>

문항 20:

Azure Data Factory(ADF)의 핵심 구성 요소(파이프라인, 활동, 데이터 세트, 연결된 서비스, 통합 런타임)에 대해 설명하시오.

Azure Data Factory(ADF)는 데이터 이동 및 변환을 오케스트레이션하는 클라우드 기반 ETL(Extract, Transform, Load) 및 데이터 통합 서비스입니다. ADF의 아키텍처는 여러 핵심 구성 요소가 상호 연결되어 데이터 기반 워크플로우를 구성하는 방식으로 이루어집니다.²⁶

1. 파이프라인 (Pipelines):

- **설명:** 파이프라인은 특정 작업을 수행하기 위한 **활동(Activities)의 논리적 그룹**입니다. 이는 데이터 이동, 변환, 제어 흐름을 포함하는 전체 엔드투엔드 워크플로우를 나타냅니다.27 예를 들어, '온프레미스 SQL Server에서 데이터를 복사하여 Azure Blob Storage에 저장한 후, Databricks Notebook을 실행하여 데이터를 변환하고, 최종 결과를 Azure Synapse Analytics에 로드한다'는 전체 과정이 하나의 파이프라인이 될 수 있습니다.
- 파이프라인 내의 활동들은 순차적으로, 병렬로, 또는 조건부로 실행될 수 있습니다.

2. 활동 (Activities):

- **설명:** 활동은 파이프라인 내에서 수행되는 **개별 처리 단계**를 나타냅니다. 각 활동은 특정 작업을 정의합니다.27 활동은 크게 세 가지 유형으로 분류됩니다.
 - **데이터 이동 활동:** 가장 일반적인 활동은 **복사 활동(Copy Activity)**으로, 원본 데이터 저장소에서 대상 데이터 저장소로 데이터를 이동시킵니다. ADF는 수많은 내장 커넥터를 통해 다양한 소스와 싱크를 지원합니다.
 - **데이터 변환 활동:** 데이터를 변환하거나 처리하는 활동입니다. 이는 매핑 데이터 흐름(Mapping Data Flows)과 같은 시각적 도구를 사용하거나, HDInsight, Azure Databricks, Azure Functions, 저장 프로시저와 같은 외부 컴퓨팅 서비스를 호출하여 수행될 수 있습니다.
 - **제어 흐름 활동:** 파이프라인의 실행 흐름을 제어합니다. 예를 들어, `ForEach` 루프, `If Condition`, `Wait`, `Execute Pipeline` 등의 활동을 사용하여 동적이고 유연한 워크플로우를 만들 수 있습니다.

3. 데이터 세트 (Datasets):

- **설명:** 데이터 세트는 활동에서 입력 또는 출력으로 사용할 **데이터를 가리키는 명명된 뷰**입니다. 이는 데이터 자체를 저장하는 것이 아니라, 데이터의 위치, 구조(스키마), 형식(예: CSV, JSON, Parquet)을 정의하는 **메타데이터**입니다.27
- 예를 들어, 'Azure Blob Storage의 `mycontainer` 컨테이너에 있는 `sales.csv` 파일'을 나타내는 데이터 세트를 정의할 수 있습니다. 복사 활동은 이 데이터 세트를 '원본'으로 참조하여 데이터를 읽고, 'Azure SQL Database의 `SalesTable`'을 나타내는 다른 데이터 세트를 '싱크'로 참조하여 데이터를 씁니다.

4. 연결된 서비스 (Linked Services):

- **설명:** 연결된 서비스는 Data Factory가 외부 리소스에 연결하는 데 필요한 **연결 정보(connection strings)**와 같습니다. 이는 데이터 저장소(예: Azure Storage, SQL Server)나 컴퓨팅 환경(예: Azure Databricks, HDInsight)에 대한 연결을 정의합니다.26
- 데이터 세트는 특정 연결된 서비스를 통해 데이터 저장소에 연결되고, 변환 활동은 특정 연결된 서비스를 통해 컴퓨팅 환경에 연결됩니다. 예를 들어, 온프레미스 SQL Server에 연결하려면 서버 이름, 데이터베이스 이름, 사용자 자격 증명 등이 포함된 연결된 서비스를 만들어야 합니다. 자격 증명은 보안을 위해 Azure Key Vault에 저장하는 것이 좋습니다.

5. 통합 런타임 (Integration Runtime - IR):

- **설명:** 통합 런타임은 Data Factory가 다양한 네트워크 환경에서 활동을 실행하기 위해 사용하는 **컴퓨팅 인프라**입니다. 이는 활동과 연결된 서비스 사이의 다리 역할을 합니다.27
- 세 가지 유형의 통합 런타임이 있습니다:
 - **Azure IR:** Azure에서 완전히 관리하는 서버리스 컴퓨팅 환경입니다. 클라우드 기반 데이터 저장소 간의 데이터 복사 및 변환 활동을 실행하는 데 사용됩니다.
 - **자체 호스팅 IR (Self-hosted IR):** 온프레미스 데이터 센터나 다른 클라우드와 같은 사설 네트워크 내에 설치되는 에이전트입니다. 클라우드와 사설 네트워크 간의 데이터 이동(예: 온프레미스 SQL Server에서 Azure Blob으로 복사)을 수행하는 데 필요합니다.
 - **Azure-SSIS IR:** 기존의 SQL Server Integration Services(SSIS) 패키지를 클라우드에서 '리프트 앤 시프트'하여 실행하기 위한 전용 컴퓨팅 환경입니다.

이 다섯 가지 구성 요소는 "데이터를 (연결된 서비스)를 통해 연결된 (데이터 세트)에서 읽어와, (통합 런타임) 위에서 실행되는 (활동)을 통해 처리하고, 그 결과를 다른 데이터 세트에 쓰는 과정을 (파이프라인)이 오케스트레이션한다"는 관계로 요약될 수 있습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/data-factory/introduction>

문항 21:

Azure Data Factory의 통합 런타임(Integration Runtime) 세 가지 유형(Azure, Self-hosted, Azure-SSIS)은 각각 어떤 경우에 사용됩니까?

Azure Data Factory(ADF)의 통합 런타임(Integration Runtime, IR)은 데이터 이동 및 활동 디스패치를 위한 컴퓨팅 브리지를 제공하는 핵심 인프라 구성 요소입니다. 어떤 IR을 사용할지는 작업이 수행되는 위치와 대상 네트워크 환경에 따라 결정됩니다. 세 가지 유형의 IR은 각각 고유한 시나리오를 해결하도록 설계되었습니다.²⁷

1. Azure Integration Runtime (Azure IR)

- **설명:** Azure에서 완전히 관리하는 서버리스 컴퓨팅 인프라입니다. 사용자는 인프라를 프로비저닝하거나 관리할 필요 없이, ADF가 필요에 따라 컴퓨팅 리소스를 동적으로 제공합니다. 이것이 기본 IR 유형입니다.
- **네트워크 환경:** 공용 인터넷을 통해 접근 가능한 클라우드 데이터 저장소 간의 통신을 담당합니다.
- **주요 사용 사례:**
 - **클라우드 간 데이터 이동:** Azure Blob Storage에서 Azure SQL Database로, 또는 AWS S3에서 Azure Data Lake Storage로 데이터를 복사하는 등, 두 개의 클라우드 기반 서비스 간에 데이터를 이동할 때 사용됩니다.
 - **서버리스 데이터 변환:** 매핑 데이터 흐름(Mapping Data Flows)과 같은 내장 데이터 변환 활동을 실행할 때 사용됩니다. ADF가 백그라운드에서 Spark 클러스터를 관리하여 시각적으로 정의된 변환 로직을 실행합니다.
 - **활동 디스패치:** Azure Databricks, HDInsight, Azure Functions 등 다른 Azure 컴퓨팅 서비스에서 실행되는 변환 활동을 디스패치(호출)하는 데 사용됩니다.

2. 자체 호스팅 통합 런타임 (Self-hosted Integration Runtime)

- **설명:** 온프레미스 데이터 센터나 Azure VNet과 같은 사설 네트워크 내의 컴퓨터에 직접 설치하고 관리하는 소프트웨어(에이전트)입니다.
- **네트워크 환경:** 클라우드 환경과 사설 네트워크 환경 간의 하이브리드 데이터 통합을 가능하게 합니다.
- **주요 사용 사례:**
 - **하이브리드 데이터 이동:** 온프레미스 SQL Server에서 Azure Synapse Analytics로 데이터를 복사하거나, Azure Blob Storage의 파일을 사설 네트워크 내의 파일 서버로 이동시키는 경우에 필수적입니다. Self-hosted IR은 사설 네트워크 내에서 데이터를 안전하게 읽어와 암호화된 채널을 통해 Azure 서비스로 전송합니다.
 - **사설 네트워크 내의 활동 실행:** Azure VNet 내에 있는 가상 머신에서 스크립트를 실행하는 등의 작업을 수행할 때 사용됩니다.
 - **게이트웨이 역할:** 온프레미스 데이터에 대한 보안 게이트웨이 역할을 하며, 데이터가 방화벽을 안전하게 통과할 수 있도록 합니다. 확장성과 고가용성을 위해 여러 노드에 설치하여 클러스터로 구성할 수 있습니다.

3. Azure-SSIS Integration Runtime (Azure-SSIS IR)

- **설명:** 기존의 **SQL Server Integration Services (SSIS)** 패키지를 클라우드에서 네이티브하게 실행하기 위해 특별히 설계된, 완전히 관리되는 Azure VM 클러스터입니다.
- **네트워크 환경:** Azure에서 프로비저닝되지만, VNet에 연결하여 온프레미스 데이터 소스에 안전하게 액세스할 수 있습니다.
- **주요 사용 사례:**
 - **SSIS 리프트 앤 시프트 (Lift-and-shift):** 온프레미스에서 수년간 개발하고 운영해 온 기존 SSIS 프로젝트와 패키지를 코드 변경 없이 클라우드로 마이그레이션하고자 할 때 사용됩니다.
 - **SSIS 현대화:** 기업은 SSIS 워크로드를 클라우드로 이전함으로써 온프레미스 SQL Server 라이선스 및 하드웨어 관리의 부담을 줄이고, 필요에 따라 컴퓨팅 성능을 확장/축소하는 클라우드의 이점을 누릴 수 있습니다. ADF 파이프라인에서 'Execute SSIS Package' 활동을 사용하여 SSIS 패키지 실행을 오케스트레이션하고 현대적인 데이터 파이프라인의 일부로 통합할 수 있습니다.

선택 가이드:

- 작업이 **순수하게 클라우드 내에서** 이루어진다면 **Azure IR**을 사용합니다.
- **온프레미스 또는 VNet**과 같은 사설 네트워크의 데이터에 액세스해야 한다면 **Self-hosted IR**을 설치하고 사용합니다.
- 기존 **SSIS 패키지**를 클라우드에서 실행하고 싶다면 **Azure-SSIS IR**을 프로비저닝합니다.

이 세 가지 IR 유형을 통해 Azure Data Factory는 거의 모든 종류의 하이브리드 및 클라우드 데이터 통합 시나리오를 지원하는 유연하고 강력한 플랫폼을 제공합니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/data-factory/concepts-integration-runtime>

문항 22:

Azure Synapse Analytics란 무엇이며, 기존의 데이터 웨어하우스 및 빅데이터 솔루션과 비교하여 어떤 통합된 가치를 제공합니까?

Azure Synapse Analytics는 **엔터프라이즈 데이터 웨어하우스**와 **빅데이터 분석**을 하나의 통합된 서비스로 결합한 **무한 분석 서비스(limitless analytics service)**입니다.²⁸ 과거에는 데이터를 분석하기 위해 데이터 웨어하우스(예: SQL Data Warehouse), 데이터 레이크, ETL 도구(예: Data Factory), 빅데이터 처리 엔진(예: Spark, Hadoop) 등 여러 개의 개별 서비스를 조합해야 했습니다. Azure Synapse는 이러한 사일로(silo)를 허물고, 데이터 엔지니어, 데이터 과학자, 비즈니스 분석가 등 모든 데이터 전문가가

Synapse Studio라는 단일 통합 작업 공간에서 협업할 수 있도록 지원합니다.²⁹

Azure Synapse Analytics의 핵심 구성 요소:

- **Synapse SQL:** 두 가지 형태의 SQL 엔진을 제공합니다.
 - **전용 SQL 풀 (Dedicated SQL Pool):** 기존의 Azure SQL Data Warehouse와 동일하며, 엔터프라이즈 데이터 웨어하우스를 위한 프로비저닝된 리소스입니다. 대규모 데이터에 대한 고성능 쿼리를 위해 MPP(Massively Parallel Processing) 아키텍처를 사용합니다.
 - **서버리스 SQL 풀 (Serverless SQL Pool):** 데이터 레이크(Azure Data Lake Storage)에 있는 파일을 직접 쿼리하기 위한 서버리스 쿼리 엔진입니다. 인프라를 프로비저닝할 필요 없이, 쿼리 실행 시 처리된 데이터 양에 따라 비용을 지불합니다. 데이터 탐색 및 논리적 데이터 웨어하우스 구축에 이상적입니다.

- **Apache Spark for Synapse:** 빅데이터 처리, 데이터 엔지니어링, 기계 학습을 위한 Apache Spark의 완전 관리형 버전입니다. Synapse Studio 내에서 Spark Notebook을 사용하여 데이터를 탐색, 준비, 변환하고 ML 모델을 학습시킬 수 있습니다.
- **Synapse 파이프라인 (Data Integration):** Azure Data Factory와 동일한 기술을 기반으로 하는 데이터 통합 및 오케스트레이션 기능입니다. 90개 이상의 커넥터를 사용하여 다양한 소스에서 데이터를 수집(ETL/ELT)하는 복잡한 데이터 파이프라인을 구축할 수 있습니다.
- **Synapse Studio:** 모든 작업을 위한 웹 기반 통합 개발 환경(IDE)입니다. 데이터 수집, 탐색, 준비, 관리, 시각화를 위한 단일 인터페이스를 제공하며, Power BI, Azure Machine Learning 등 다른 서비스와도 긴밀하게 통합됩니다.

통합된 가치와 기존 솔루션과의 차이점:

Azure Synapse가 제공하는 가장 큰 가치는 '통합'을 통해 분석 프로젝트의 복잡성과 시간을 단축하는 데 있습니다.

1. **사일로 해소 및 데이터 이동 최소화:** 기존 방식에서는 데이터 웨어하우스용 데이터와 빅데이터 분석용 데이터를 별도의 시스템에서 관리해야 했습니다. Synapse는 데이터 레이크와 데이터 웨어하우스를 긴밀하게 통합합니다. 예를 들어, Spark 풀에서 데이터 레이크의 Parquet 파일을 처리하여 정제된 후, 간단한 T-SQL 명령으로 전용 SQL 풀에 고속으로 로드 (PolyBase/COPY)할 수 있습니다. 또한, 서버리스 SQL 풀을 사용하면 데이터를 물리적으로 이동하지 않고도 데이터 레이크의 데이터를 즉시 쿼리할 수 있습니다.
2. **단일 통합 환경(Synapse Studio):** 과거에는 데이터 엔지니어는 Data Factory에서, 데이터 과학자는 Databricks에서, 데이터 분석가는 SQL Server Management Studio(SSMS)에서 각각 작업해야 했습니다. Synapse Studio는 이 모든 역할을 하나의 UI로 통합하여 협업을 촉진하고 개발 생산성을 높입니다. 데이터 엔지니어가 파이프라인을 만들고, 데이터 과학자가 Spark Notebook으로 모델을 개발하며, 분석가가 SQL 스크립트를 실행하고 Power BI 보고서를 만드는 모든 활동이 한 곳에서 이루어집니다.²⁹
3. **유연한 쿼리 모델:** 워크로드의 특성에 따라 **프로비저닝된 리소스(전용 SQL 풀)**와 **서버리스(서버리스 SQL 풀, Spark 풀)** 모델을 자유롭게 선택하고 조합할 수 있습니다. 예측 가능한 고성능 쿼리가 필요할 때는 전용 SQL 풀을, 데이터 탐색이나 간헐적인 쿼리에는 서버리스 SQL 풀을 사용하여 비용을 최적화할 수 있습니다.
4. **엔드투엔드 분석 솔루션 가속화:** Microsoft의 데이터 전략은 분산된 서비스를 통합 플랫폼으로 수렴시키는 것입니다. 이는 분석 솔루션 개발에 필요한 'Time to Insight'를 줄이는 것을 목표로 합니다. 전통적인 분석 워크플로우는 여러 서비스를 연결하고 데이터를 이동시키는 데 많은 시간과 노력이 소요되었습니다. Synapse는 데이터 수집, 저장, 처리, 분석, 시각화에 이르는 전체 라이프사이클을 단일 플랫폼에서 지원함으로써, 아이디어에서 실제 비즈니스 가치를 창출하기까지의 시간을 획기적으로 단축합니다.

결론적으로, Azure Synapse Analytics는 개별 분석 구성 요소의 단순한 집합이 아니라, 이들을 유기적으로 통합하여 데이터에서 가치를 더 빠르고 효율적으로 추출할 수 있도록 설계된 차세대 분석 플랫폼입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/synapse-analytics/overview-what-is>

문항 23:

Azure Synapse Analytics의 서버리스 SQL 풀과 전용 SQL 풀의 차이점은 무엇이며, 각각 어떤 시나리오에 더 적합합니까?

Azure Synapse Analytics는 데이터 쿼리 및 분석을 위해 두 가지 다른 SQL 리소스 모델인 **서버리스 SQL 풀(Serverless SQL pool)**과 **전용 SQL 풀(Dedicated SQL pool)**을 제공합니다. 이 둘은 아키텍처, 비용 모델, 성능 특성 및 주요 사용 사례에서 근본적인 차이가 있습니다.²⁸

서버리스 SQL 풀 (Serverless SQL pool):

- **아키텍처 및 개념:** 이름에서 알 수 있듯이, 인프라를 프로비저닝하거나 관리할 필요가 없는 **쿼리 서비스**입니다. 사용자가 쿼리를 실행하면 Azure가 자동으로 컴퓨팅 리소스를 할당하고, 쿼리가 완료되면 리소스를 해제합니다. 이 풀은 Azure Data Lake Storage(ADLS Gen2), Azure Cosmos DB, Dataverse 등에 저장된 데이터를 **직접 쿼리**하는 데 사용됩니다. 데이터를 미리 Synapse로 로드할 필요가 없습니다.
- **비용 모델:** 처리된 데이터의 양(TB)에 따라 비용을 지불하는 종량제 모델입니다. 쿼리를 실행하지 않으면 컴퓨팅 비용이 발생하지 않습니다.
- **성능:** 쿼리 복잡성, 데이터 양, 파일 형식 등에 따라 성능이 달라집니다. 데이터 탐색 및 임시 쿼리에 적합하지만, 일관된 고성능을 보장하지는 않습니다.
- **주요 사용 사례:**
 - **데이터 탐색 및 검색:** 데이터 레이크에 있는 새로운 데이터 세트의 내용을 신속하게 파악하고 스키마를 추론하는 데 이상적입니다.
 - **논리적 데이터 웨어하우스 (Logical Data Warehouse):** 데이터를 물리적으로 이동하거나 복제하지 않고, 데이터 레이크의 원시 데이터를 기반으로 뷰(View)를 생성하여 BI 도구(예: Power BI)에서 직접 분석할 수 있도록 합니다.
 - **간단한 데이터 변환:** `CETAS (CREATE EXTERNAL TABLE AS SELECT)` 와 같은 T-SQL 명령을 사용하여 데이터 레이크 내에서 간단한 데이터 변환 및 정제 작업을 수행할 수 있습니다.
 - **예측 불가능한 또는 간헐적인 쿼리 워크로드:** 사용량이 많지 않거나 예측하기 어려운 쿼리 패턴을 가진 시나리오에서 비용 효율적입니다.

전용 SQL 풀 (Dedicated SQL pool):

- **아키텍처 및 개념:** 엔터프라이즈 데이터 웨어하우징을 위한 **대규모 병렬 처리(Massively Parallel Processing, MPP)** 아키텍처를 사용하는 프로비저닝된 리소스입니다. 데이터를 여러 컴퓨팅 노드에 분산 저장하고 쿼리를 병렬로 처리하여 대규모 데이터 세트에 대한 고성능 분석을 제공합니다. 데이터를 쿼리하기 전에 먼저 전용 SQL 풀로 **로드(load)**해야 합니다.
- **비용 모델:** DWU(Data Warehouse Units)라는 단위로 프로비저닝된 컴퓨팅 성능에 대해 **시간당** 비용을 지불합니다. 사용하지 않을 때는 풀을 일시 **중지(pause)**하여 컴퓨팅 비용을 절약할 수 있습니다 (이 경우 스토리지 비용만 발생).
- **성능:** 일관되고 예측 가능한 고성능을 제공합니다. 복잡한 조인, 집계, 윈도우 함수 등을 포함하는 까다로운 쿼리에 최적화되어 있습니다.
- **주요 사용 사례:**
 - **엔터프라이즈 데이터 웨어하우징:** 정형 데이터를 저장하고 BI 및 보고를 위한 단일 진실 공급원(single source of truth)을 구축하는 전통적인 데이터 웨어하우스 워크로드.
 - **까다로운 BI 대시보드 및 보고:** Power BI와 같은 도구에서 빠른 응답 시간이 요구되는 대화형 대시보드를 지원합니다.
 - **예측 가능한 고성능 요구사항:** 일관된 쿼리 성능이 비즈니스에 중요한 미션 크리티컬 분석 애플리케이션.
 - **대규모 데이터에 대한 복잡한 분석 쿼리:** 수십억 개의 행을 가진 테이블을 조인하고 집계해야 하는 경우.

요약 비교:

특징	서버리스 SQL 풀	전용 SQL 풀
리소스 모델	서버리스 (온디맨드)	프로비저닝 (MPP)
데이터 위치	데이터 레이크의 데이터를 직접 쿼리	데이터를 풀 내부로 로드 필요
비용 모델	처리된 데이터 양(TB) 기준	프로비저닝된 컴퓨팅(DWU) 시간 기준
주요 목적	데이터 탐색, 논리적 DW	엔터프라이즈 DW, 고성능 BI
성능	가변적, 임시 쿼리에 적합	일관적, 예측 가능한 고성능
관리	관리 불필요	크기 조정, 일시 중지/재개 관리 필요

결론적으로, 두 SQL 풀은 상호 보완적인 관계에 있습니다. **서버리스 SQL 풀**은 데이터 레이크의 데이터를 빠르고 유연하게 탐색하고 활용하는 데 사용되며, **전용 SQL 풀**은 정제되고 구조화된 데이터를 기반으로 최고의 성능과 동시성을 요구하는 엔터프라이즈급 분석을 제공하는 데 사용됩니다. 현대적인 데이터 아키텍처에서는 이 두 가지를 함께 사용하여 다양한 분석 요구사항을 충족시키는 경우가 많습니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/synapse-analytics/sql/overview-architecture>

문항 24:

Azure Databricks란 무엇이며, Azure Synapse Analytics와 어떤 관계에 있습니까?

Azure Databricks는 Apache Spark를 기반으로 구축된, 통합되고 최적화된 **데이터 분석 및 AI 플랫폼**입니다. 이는 원래 Apache Spark의 창시자들이 설립한 Databricks사와 Microsoft 간의 파트너십을 통해 제공되는 1등급(first-party) Azure 서비스입니다. Azure Databricks는 데이터 엔지니어링, 데이터 과학, 기계 학습을 위한 협업 환경을 제공하며, 대규모 데이터 처리와 고급 분석 워크로드를 간소화하는 데 중점을 둡니다.³⁰

Azure Databricks의 핵심 기능:

- **최적화된 Apache Spark 엔진:** Azure Databricks는 표준 Apache Spark보다 향상된 성능과 안정성을 제공하는 최적화된 런타임을 사용합니다.
- **협업 작업 공간 (Collaborative Workspace):** 데이터 과학자, 엔지니어, 분석가들이 Notebook, 실험, 모델 등을 공유하고 함께 작업할 수 있는 대화형 작업 공간을 제공합니다.
- **Delta Lake:** ACID 트랜잭션, 스키마 적용, 시간 여행(time travel)과 같은 안정성 기능을 데이터 레이크에 추가하는 오픈 소스 스토리지 계층입니다. 이를 통해 데이터 레이크를 기반으로 신뢰할 수 있는 'Lakehouse' 아키텍처를 구축할 수 있습니다.
- **통합된 ML 수명 주기 (MLflow):** 기계 학습 실험 추적, 모델 패키징, 배포 및 레지스트리 관리를 위한 오픈 소스 플랫폼인 MLflow가 기본적으로 통합되어 있어 엔드투엔드 MLOps를 지원합니다.
- **Databricks SQL:** 데이터 웨어하우징 및 BI 워크로드를 위한 고성능 SQL 분석 환경을 제공하여, 분석가들이 익숙한 SQL을 사용하여 Lakehouse의 데이터에 직접 쿼리할 수 있도록 합니다.

Azure Synapse Analytics와의 관계:

Azure Databricks와 Azure Synapse Analytics는 일부 기능이 겹치지만(둘 다 Spark와 SQL 분석을 제공), 서로 다른 강점과 설계 철학을 가진 서비스이며, 종종 함께 사용되기도 합니다.

특징	Azure Databricks	Azure Synapse Analytics
핵심 강점	고급 분석 및 AI/ML에 최적화된 Apache Spark 플랫폼	데이터 통합, 데이터 웨어하우징, 빅데이터 분석을 결합한 통합 분석 서비스
주요 사용자	데이터 과학자, ML 엔지니어, 데이터 엔지니어	데이터 엔지니어, 데이터베이스 관리자(DBA), BI 분석가
Spark 환경	가장 진보되고 성능이 최적화된 Spark 환경 제공. 최신 Spark 버전을 빠르게 지원.	완전 관리형 Spark 풀 제공. 데이터 엔지니어링 및 기본 ML 작업에 적합.
SQL 환경	Databricks SQL (Lakehouse 데이터에 대한 고성능 쿼리)	전용 SQL 풀 (MPP 기반 엔터프라이즈 DW) 및 서버리스 SQL 풀 (데이터 레이크 쿼리)
통합	다양한 Azure 서비스(ADLS, Event Hubs 등)와 긴밀하게 통합되지만, 플랫폼 자체는 Spark 중심.	Synapse Studio라는 단일 UI 내에서 데이터 통합 (ADF), SQL, Spark, Power BI를 매우 긴밀하게 통합.
사용 시나리오	- 복잡한 ML 모델 개발 및 대규모 학습 - 스트리밍 데이터 분석 - 유전체학, 추천 엔진 등 고급 분석 - Delta Lake 기반의 Lakehouse 구축	- 기존 데이터 웨어하우스를 현대화 - 다양한 소스로부터 데이터 수집 및 ETL/ELT 파이프라인 구축 - BI 및 보고를 위한 통합 데이터 플랫폼 구축

상호 보완적 관계:

두 서비스는 경쟁 관계이기도 하지만, 많은 엔터프라이즈 아키텍처에서 함께 사용되어 각자의 강점을 활용합니다.

- **Databricks for ML, Synapse for Serving:** Azure Databricks의 강력한 Spark 및 MLflow 환경에서 복잡한 기계 학습 모델을 개발하고 학습시킨 후, 모델의 예측 결과를 Azure Synapse Analytics의 전용 SQL 풀에 저장하여 Power BI와 같은 BI 도구를 통해 비즈니스 사용자에게 제공할 수 있습니다.
- **Synapse for Ingestion, Databricks for Transformation:** Azure Synapse 파이프라인을 사용하여 다양한 온프레미스 및 클라우드 소스에서 데이터를 수집하여 데이터 레이크에 저장합니다. 그런 다음, Azure Databricks를 사용하여 이 원시 데이터에 대해 복잡한 변환 및 정제 작업을 수행하고, 처리된 데이터를 다시 데이터 레이크나 Synapse에 저장합니다.

결론적으로, **Azure Databricks**는 최고의 성능을 자랑하는 Spark 기반의 고급 분석 및 AI 워크로드에 중점을 둔 'Best-of-breed' 플랫폼에 가깝습니다. 반면, **Azure Synapse Analytics**는 다양한 분석 기능을 하나의 관리형 서비스로 통합하여 엔드 투엔드 분석 솔루션의 개발 및 관리를 간소화하는 'All-in-one' 플랫폼에 가깝습니다. 선택은 조직의 기존 기술 스택, 주요 사용 사례, 그리고 사용자 그룹의 전문성에 따라 달라집니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/databricks/introduction/>

문항 25:

Azure Machine Learning이란 무엇이며, 머신러닝 모델 개발의 전체 수명 주기(MLOps)를 어떻게 지원합니까?

Azure Machine Learning(Azure ML)은 데이터 과학자와 개발자가 머신러닝 모델을 **빌드, 학습, 배포, 관리**할 수 있도록 지원하는 포괄적인 클라우드 기반 플랫폼입니다. 이는 단순히 모델을 학습시키는 라이브러리나 도구가 아니라, 아이디어 구상부터 프로덕션 운영에 이르기까지 머신러닝 프로젝트의 전체 수명 주기를 가속화하고 관리하기 위한 통합 환경을 제공합니다.³²

Azure Machine Learning의 핵심 기능:

- **Azure Machine Learning 스튜디오:** 모든 ML 작업을 위한 중앙 집중식 웹 포털입니다. 데이터 세트 관리, Notebook 실행, 실험 추적, 모델 배포 등을 위한 시각적 인터페이스와 코드 기반 도구를 모두 제공합니다.³³
- **데이터 및 기능 관리:** 데이터 세트와 데이터 저장소를 등록하고 버전을 관리하여 데이터의 출처와 재현성을 보장합니다. 피처 스토어(Feature Store)를 통해 여러 프로젝트에서 피처를 공유하고 재사용할 수 있습니다.
- **통합 Notebooks:** 스튜디오 내에서 직접 Jupyter Notebook을 실행할 수 있는 관리형 컴퓨팅 인스턴스를 제공하여, 데이터 탐색 및 모델 개발을 위한 친숙한 환경을 제공합니다.
- **다양한 학습 옵션:**
 - **자동화된 ML (Automated ML):** 코딩 없이도 데이터에 가장 적합한 알고리즘과 하이퍼파라미터를 자동으로 찾아 모델을 생성합니다.
 - **디자이너 (Designer):** 드래그 앤 드롭 인터페이스를 사용하여 시각적으로 ML 파이프라인을 구축합니다.
 - **SDK/CLI:** Python SDK 또는 Azure CLI를 사용하여 코드 중심의 방식으로 학습 및 배포 워크플로우를 완벽하게 제어합니다.
- **확장 가능한 컴퓨팅:** 필요에 따라 강력한 GPU 또는 CPU 클러스터를 온디맨드로 프로비저닝하여 대규모 모델 학습을 가속화할 수 있습니다.

MLOps (Machine Learning Operations) 지원 방식:

MLOps는 DevOps의 원칙을 머신러닝 시스템에 적용하여 모델 개발과 운영을 통합하고 자동화하는 관행입니다. Azure ML은 MLOps의 핵심 단계를 지원하는 다양한 기능을 제공합니다.

1. 재현 가능한 학습 (Reproducible Training):

- **ML 파이프라인:** 데이터 준비, 학습, 검증, 등록 등 전체 학습 워크플로우를 재사용 가능한 구성 요소로 구성된 파이프라인으로 정의할 수 있습니다. 이를 통해 전체 프로세스를 자동화하고 일관성을 보장합니다.³²
- **환경 (Environments):** 학습에 사용된 Python 패키지, Docker 이미지, 환경 변수 등을 캡슐화하여 어떤 컴퓨팅 환경에서든 동일한 조건으로 학습을 재현할 수 있도록 합니다.
- **실험 추적 (Experiment Tracking):** 모든 학습 실행(run)에 대한 메트릭, 파라미터, 로그, 생성된 모델 파일 등을 자동으로 기록하고 비교할 수 있습니다. 이를 통해 어떤 조건에서 최고의 모델이 나왔는지 추적하고 감사할 수 있습니다.

2. 모델 관리 및 버전 관리:

- **모델 레지스트리 (Model Registry):** 학습된 모델을 중앙 레지스트리에 등록하고 버전을 관리합니다. 각 모델 버전은 학습 실행, 소스 코드, 성능 메트릭과 연결되어 완전한 추적성을 제공합니다.

3. 자동화된 배포 (Automated Deployment):

- **관리형 엔드포인트 (Managed Endpoints):** 학습된 모델을 실시간 추론(real-time scoring)을 위한 온라인 엔드포인트나 배치 추론(batch scoring)을 위한 배치 엔드포인트에 쉽게 배포할 수 있습니다. Azure ML이 기본 인프라(컴퓨팅, 네트워킹, 스케일링)를 모두 관리해 줍니다.
- **CI/CD 통합:** Azure DevOps나 GitHub Actions와 같은 CI/CD 도구와 통합하여, 새로운 코드가 커밋되거나 새로운 모델이 등록될 때마다 자동으로 학습 파이프라인을 트리거하고, 검증을 통과한 모델을 스테이징 또는 프로덕션 환경에 배포하는 전체 프로세스를 자동화할 수 있습니다.³³

4. 모니터링 및 재학습:

- **모델 모니터링:** 배포된 모델의 성능(예: 대기 시간, 요청 속도)과 입력 데이터의 드리프트(drift, 프로덕션 데이터의 분포가 학습 데이터와 달라지는 현상)를 모니터링합니다.
- **재학습 트리거:** 데이터 드리프트가 감지되거나 모델 성능이 저하되면, 자동으로 재학습 파이프라인을 트리거하여 새로운 데이터로 모델을 업데이트하고 다시 배포하는 피드백 루프를 구축할 수 있습니다.

5. 책임감 있는 AI (Responsible AI):

- **해석 가능성 및 공정성:** 모델이 왜 특정 예측을 했는지 설명하는 도구(Interpretability)와 모델의 편향을 평가하고 완화하는 도구(Fairness)를 제공하여 투명하고 공정한 AI 시스템을 구축하도록 돕습니다.³³

결론적으로, Azure Machine Learning은 단일 모델을 만드는 것을 넘어, 견고하고 재현 가능하며 자동화된 방식으로 머신러닝 솔루션을 프로덕션에 제공하고 지속적으로 유지 관리하는 데 필요한 모든 도구와 프레임워크를 제공하는 엔드투엔드 MLOps 플랫폼입니다.

관련 링크: <https://learn.microsoft.com/ko-kr/azure/machine-learning/overview-what-is-azure-machine-learning>